Computer Vision

Summary of lectures

Prof. Irene Amerini
Academic year 2024/2025

Contents

1 Image processing **1**
- 1.1 Point processing . . . 1
  - 1.1.1 Gamma correction . . . 2
  - 1.1.2 Histogram equalization . . . 2
- 1.2 Linear filtering . . . 3
  - 1.2.1 Cross-correlation and convolution . . . 3
  - 1.2.2 Padding and boundaries . . . 3
  - 1.2.3 Examples of linear filters . . . 4
- 1.3 Nonlinear filtering . . . 5
  - 1.3.1 Bilateral filtering . . . 6
  - 1.3.2 Median filter . . . 6
  - 1.3.3 Thresholding . . . 6
- 1.4 Image scaling . . . 7
  - 1.4.1 Sub-sampling and aliasing . . . 7
  - 1.4.2 Upsampling and interpolation . . . 7
- 1.5 Image derivatives and edge detection . . . 8
  - 1.5.1 The image gradient . . . 8
  - 1.5.2 Second-order derivatives: the Laplacian . . . 9
  - 1.5.3 The Sobel operator . . . 11
  - 1.5.4 The Canny edge detection pipeline . . . 11

2 Image pyramids **12**
- 2.1 Gaussian pyramids . . . 12
- 2.2 Laplacian pyramids . . . 12
  - 2.2.1 Reconstruction . . . 13
  - 2.2.2 Summary . . . 13
- 2.3 Applications of pyramids . . . 13
  - 2.3.1 Image blending . . . 14

3 Frequency domain **15**
- 3.1 The frequency domain and Fourier series . . . 15
  - 3.1.1 Building signals from sinusoids . . . 15
- 3.2 The Fourier transform . . . 15
  - 3.2.1 Mathematical definition . . . 16
  - 3.2.2 The discrete fourier transform (DFT) . . . 16
- 3.3 Fourier analysis in 2D . . . 16
- 3.4 Filtering in the frequency domain . . . 17
  - 3.4.1 The convolution theorem . . . 17
- 3.5 Sampling . . . 19
- 3.6 Hybrid images . . . 19

4 Local features **21**
- 4.1 Introduction to local features . . . 21
- 4.2 Harris corner detector . . . 21
  - 4.2.1 The mathematics of corner detection . . . 21
  - 4.2.2 Interpreting the second moment matrix . . . 22
  - 4.2.3 The harris operator and algorithm . . . 23
  - 4.2.4 Properties and limitations . . . 23
- 4.3 Scale-invariant blob detection . . . 24

5 Local descriptors **26**
5.1 Designing feature descriptors . . . 26
5.1.1 Evolution of a descriptor . . . 26
5.1.2 Comparing histograms . . . 26
5.2 Histogram of Oriented Gradients (HOG) descriptor . . . 27
5.2.1 HOG pipeline . . . 27

6 Scale-Invariant Feature Transform (SIFT) **29**
6.1 SIFT Detector . . . 29
6.1.1 Scale-space construction . . . 29
6.1.2 Scale-space extrema detection . . . 30
6.1.3 Orientation assignment . . . 30
6.2 SIFT descriptor . . . 31
6.3 Feature matching . . . 32
6.3.1 Lowe's ratio test . . . 32
6.4 Evaluating matching performance . . . 32
6.5 SURF: Speeded-Up Robust Features . . . 33
6.5.1 Hessian-based interest point detection . . . 33
6.5.2 SURF descriptor . . . 34
6.6 Binary feature descriptors . . . 34
6.6.1 General approach . . . 34
6.6.2 ORB: Oriented FAST and Rotated BRIEF . . . 35

7 Transformation and alignment **36**
7.1 Image warping . . . 36
7.2 Types of 2D transformations . . . 36
7.2.1 Linear transformations . . . 36
7.2.2 Affine transformations . . . 37
7.2.3 Projective transformations (homographies) . . . 37
7.3 Computing transformations . . . 38
7.3.1 Computing a linear transformation . . . 39
7.3.2 Computing an affine transformation . . . 39
7.3.3 Computing a homography . . . 40
7.4 Robust estimation with RANSAC . . . 41
7.5 Image warping implementation and blending . . . 41
7.5.1 Forward vs. inverse warping . . . 41
7.5.2 Image blending . . . 42

8 Recognition **43**
8.1 Recognition tasks . . . 43
8.2 Image classification . . . 43
8.2.1 Bag-of-Words (BoW) model . . . 43
8.3 Convolutional Neural Networks (CNNs) . . . 43
8.3.1 Output and loss function . . . 43
8.3.2 Resnet . . . 43
8.4 Semantic segmentation . . . 44
8.5 Object detection . . . 44
8.5.1 Two-stage detectors . . . 44
8.5.2 Single-stage detectors . . . 44
8.5.3 Feature Pyramid Networks (FPN) . . . 44
8.6 Extending the recognition framework . . . 44
8.6.1 Instance segmentation with Mask R-CNN . . . 44

8.6.2 Panoptic segmentation 45

9 Optical flow **46**
9.1 Introduction to optical flow 46
9.1.1 The aperture problem 47
9.2 Estimating optical flow 47
9.2.1 The brightness constancy equation 48
9.3 Lucas-Kanade optical flow 48
9.3.1 Lucas-Kanade solution 49
9.3.2 Handling large motions 49
9.4 Horn-Schunck optical flow 50
9.4.1 Robust regularization 50

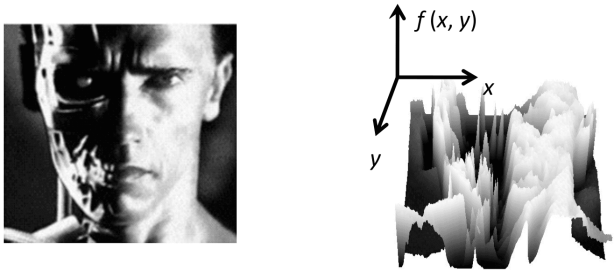
10 Camera calibration **51**
10.1 The camera model 51
10.1.1 The pinhole camera 51
10.1.2 From image plane to pixels 52
10.1.3 The camera matrix 52
10.2 Camera calibration 54
10.2.1 Extracting intrinsic and extrinsic parameters 54
10.3 Stereo vision 55
10.3.1 Depth from disparity 55
10.3.2 Stereo matching 55
10.3.3 Stereo as energy minimization 56

11 Epipolar geometry **57**
11.1 Introduction 57
11.1.1 The epipolar constraint 57
11.2 The essential matrix 58
11.2.1 Derivation 58
11.2.2 Properties of the essential matrix 59
11.3 The fundamental matrix 59
11.3.1 Derivation 59
11.3.2 Properties of the fundamental matrix 60
11.4 Estimating the fundamental matrix 60
11.4.1 The 8-Point algorithm 60
11.4.2 The normalized 8-Point algorithm 60
11.5 Triangulation 61
11.5.1 Linear triangulation method 61

1 Image processing

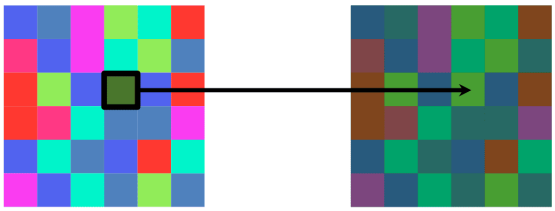
Introduction An image can be interpreted:

- in the **discrete domain** as a grid of intensity values in $[0, 2^{\text{bits}}]$.
- in the **continuous domain**, where a grayscale image can be thought of as a function $f : \mathbb{R}^2 \to \mathbb{R}$, where $f(x, y)$ gives the intensity at the spatial position (x, y). A *digital image* is just a discrete, sampled, and quantized version of this continuous function.



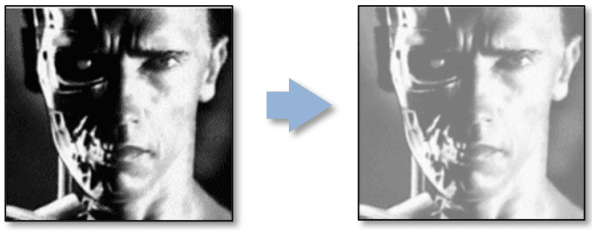
1.1 Point processing

Point operators modify the value of each pixel **independently of its neighbors**. The new value of a pixel at position (x, y) depends only on the original value at that same position.

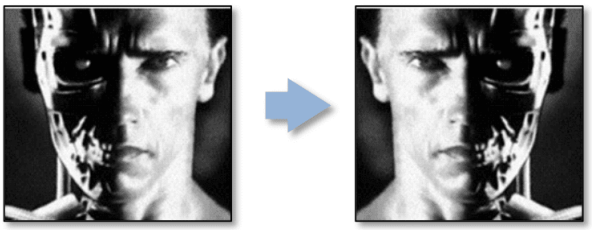


Examples Simple mathematical operations can be applied to every pixel to achieve different effects:

- **Brightness**: $g(x, y) = f(x, y) + b$. Adding a constant b lightens or darkens the image.



- **Contrast**: $g(x, y) = a \cdot f(x, y)$. Multiplying by a constant a increases ($a > 1$) or decreases ($a < 1$) the contrast.
- **Inversion**: $g(x, y) = 255 - f(x, y)$ for an 8-bit image.
- **Horizontal flip**: $g(x, y) = f(-x, y)$

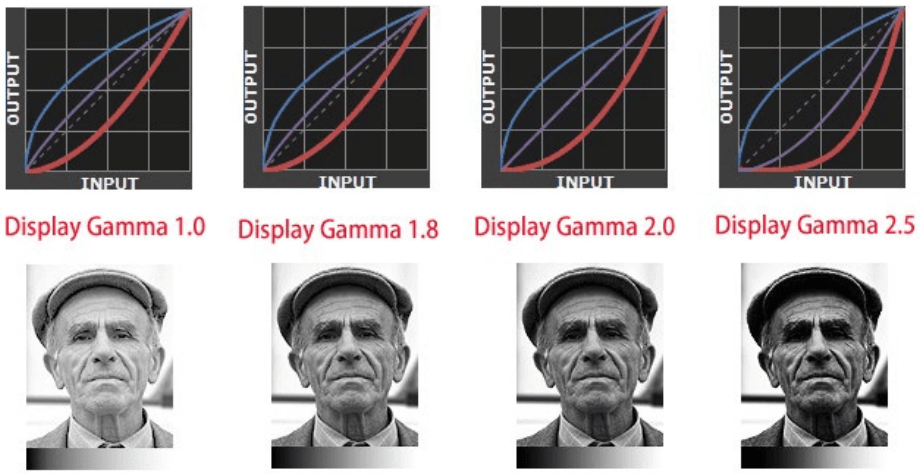


1.1.1 Gamma correction

Gamma correction **adjusts the brightness** of an image displayed on a screen by making the image more natural to the eye:

$$V_{\text{out}} = A \cdot V_{\text{in}}^{\gamma}$$

where V_{in} is the input voltage, V_{out} is the adjusted output, A is constant, and typically $\gamma = 2.2$.



1.1.2 Histogram equalization

HE adjusts the global constrast by redistributing intensity values to produce a flatter histogram, making details more visible.

1. **Compute the histogram** by counting the number of pixels at each intensity level.

$$p(r_k) = \frac{\#\text{pixels with intensity } r_k}{\#\text{pixels}}$$

2. **Compute the CDF** of the histogram which maps each input intensity to a value in $[0, 1]$.

$$\text{CDF}(r_k) = \sum_{i=0}^{k} p(r_i)$$

3. Scale the CDF to the full intensity range and use it as a lookup table to transform the original pixel values to their new equalized values.

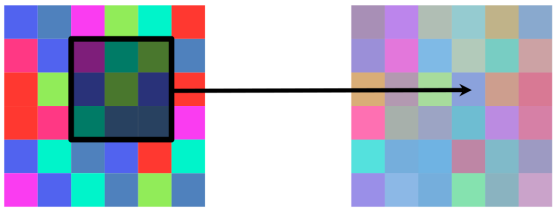
$$r'_k = \text{round}\left(\frac{(\text{CDF}(r_k) - \text{CDF}_{\min})(L-1)}{N - \text{CDF}_{\min}}\right)$$

where r'_k is the new intensity, L is the maximum pixel intensity, N is the total number of pixels, and $\text{CDF}_{\min}$ is the minimum non-zero value of the CDF.

Local Histogram Equalization Compute a transformation for each pixel based on the histogram of a small neighborhood around it, enhancing local details at the expense of computational complexity. *Adaptive Histogram Equalization* (**AHE**) is a common example.

1.2 Linear filtering

Filtering creates a new image where each pixel's value is a **function of its neighbors** in the original image.



Specifically, *linear filtering* computes each new pixel as a **linear combination** (a weighted sum) of its neighbors, where the set of weights is called the **kernel** or **filter**.

1.2.1 Cross-correlation and convolution

These are the two primary linear filtering operations.

Cross-correlation The output $G[i, j]$ is the sum of element-wise products between the kernel H and the image neighborhood F centered at (i, j).

$$G[i, j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u, v]F[i + u, j + v]$$

Convolution This is the same as cross-correlation, except the kernel H is flipped both horizontally and vertically before being applied.

$$G[i, j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u, v]F[i - u, j - v]$$

Properties of convolution

Convolution is a multiplication-like operation with several useful mathematical properties that cross-correlation lacks:

- **Commutative**: $a * b = b * a$
- **Associative**: $(a * b_1) * b_2 = a * (b_1 * b_2)$. This is very useful, as applying two filters in sequence is equivalent to convolving them into a single filter first.
- It distributes over addition.

In practice, if the kernel is symmetric, convolution and cross-correlation are identical.

1.2.2 Padding and boundaries

When the kernel is centered on a pixel near the image edge, the kernel window may fall off the boundary. We need a strategy to handle this.

- **Zero**: Pad the outside with 0.
- **Constant**: Pad with a constant value c.
- **Clamp/replicate**: Replicate the edge.

- **Reflection/mirror**: Mirror inwards.
- **Symmetric**: Mirror the edge.

Output size

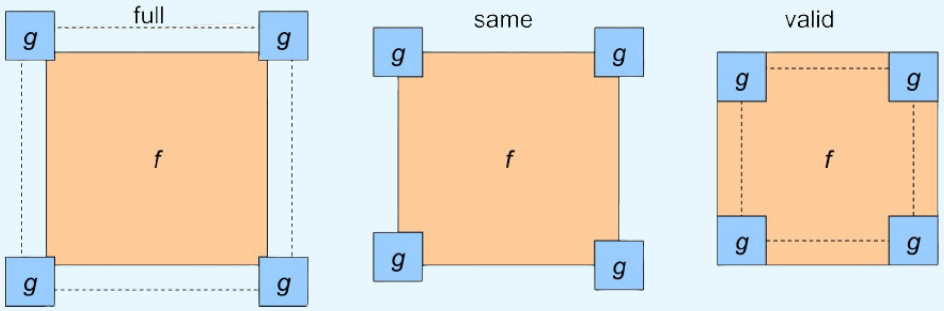
The size of the output image depends on the padding strategy.

- **'full'**: Pad such that every pixel of the input is visited by every element of the kernel. The output is larger than the input.
- **'same'**: Pad such that the output is the same size as the input.

$$\dim_{\text{out}} = \left\lceil \frac{\dim_{\text{in}}}{S} \right\rceil + 1$$

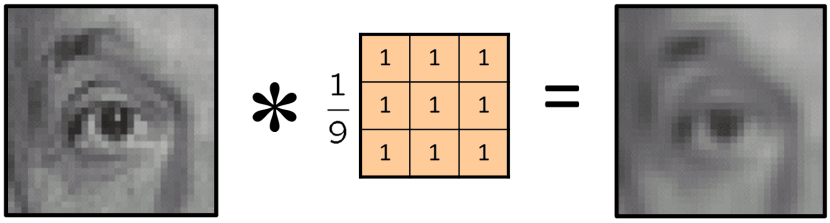
- **'valid'**: No padding. The output is smaller than the input.

$$\dim_{\text{out}} = \left\lfloor \frac{\dim_{\text{in}} - \dim_K}{S} \right\rfloor + 1$$



1.2.3 Examples of linear filters

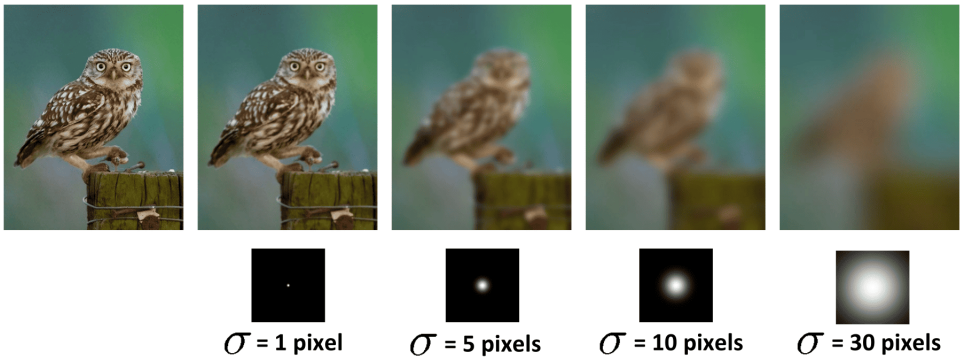
Box filter (mean filter) This filter replaces each pixel with the average value of its neighbors in a rectangular window. The kernel consists of all 1s, normalized by the number of elements (e.g., divided by 9 for a 3x3 kernel). It results in image blurring.



Gaussian filter This is a more sophisticated low-pass filter that uses a kernel with weights drawn from a 2D Gaussian distribution. Pixels closer to the center receive higher weights.

$$G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

The parameter σ controls the amount of blurring. Gaussian filters produce a smoother, more natural-looking blur than a box filter.



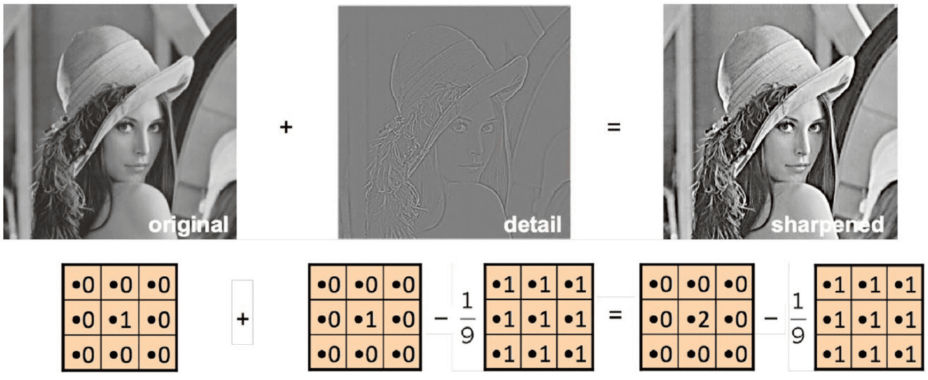
Gaussian properties

- Convolving a Gaussian with another Gaussian produces a new, wider Gaussian.
- A 2D Gaussian kernel is **separable**, meaning the 2D convolution can be performed as two 1D convolutions (one horizontal, one vertical), which is much more computationally efficient.

Sharpening filter (unsharp masking) A sharpening effect can be achieved by amplifying the image's details. Details can be extracted by subtracting a blurred version of the image from the original. These details are then added back to the original image.

$$F_{\text{sharpened}} = F_{\text{original}} + \alpha(F_{\text{original}} - F_{\text{blurred}})$$

where α is the sharpening strength ($F_{\text{sharpened}} = F_{\text{original}}$ for $\alpha = 0$).



This can be implemented with a single filter that approximates a scaled impulse minus a Gaussian.

1.3 Nonlinear filtering

Nonlinear filters also perform neighborhood operations, but the new pixel value is not a linear combination of its neighbors.

1.3.1 Bilateral filtering

This is an advanced edge-preserving smoothing filter. A standard Gaussian filter averages pixels regardless of their content, causing blurring across object boundaries.

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

The bilateral filter solves this by weighting neighboring pixels based on both their **spatial distance** and their **intensity difference**.

$$h[m,n] = \frac{1}{W_{mn}} \sum_{k,l} g[k,l] r_{mn}[k,l] f[m+k, n+l]$$

where:

- $g[k,l]$ is the *spatial Gaussian kernel*, weighting pixels based on their Euclidean distance from the center pixel (m,n).
- $r_{mn}[k,l]$ is the *range kernel* (or *intensity kernel*), weighting pixels based on their intensity difference from the center pixel $f[m,n]$.
- $W_{mn} = \sum_{k,l} g[k,l] r_{mn}[k,l]$ is the sum of all weights and works as a normalizing factor.

A pixel contributes to the average only if it is both spatially close and photometrically similar, meaning the bilateral filter smooths out noise within flat regions but preserves sharp edges, where neighbors have very different intensity values.

Its applications include high-quality denoising, photo retouching, and creating cartoon effect.

1.3.2 Median filter

This filter replaces each pixel with the median value of all pixels in its neighborhood.

Median vs. mean filtering

The median filter removes **salt-and-pepper noise**: *outliers* are ignored by the median, while the mean spreads noise into surrounding pixels and blur edges.

Noisy image | Filtered by Mean Filter | Filtered by Median Filter

1.3.3 Thresholding

This is a simple non-linear operation that converts a grayscale image into a binary image. If a pixel's intensity is above a certain threshold value, it is set to white (255); otherwise, it is set to black (0).

$$g(m,n) = \begin{cases} 255 & f(m,n) > A \\ 0 & \text{otherwise} \end{cases}$$

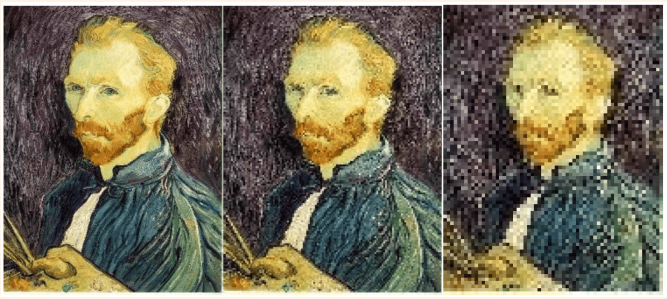
1.4 Image scaling

1.4.1 Sub-sampling and aliasing

To reduce an image's size, a naive approach is to **sub-sample** it by throwing away rows and columns.

Aliasing

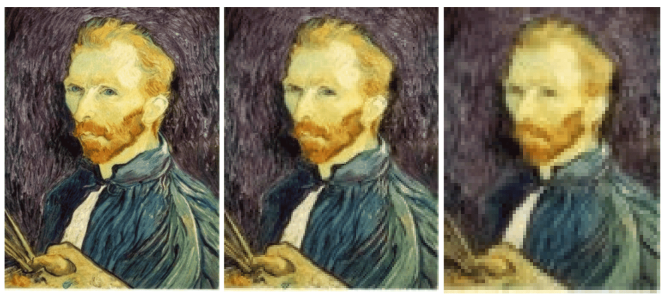
Simple sub-sampling can lead to **aliasing**, an effect where fine details and patterns in the original image are distorted into new, incorrect patterns in the downsized image. This happens when the **sampling rate is too low to capture the highest frequencies in the image**.



To avoid this, the sampling rate must be at least twice the maximum frequency in the image (the **Nyquist rate**).

$$f_s \geq 2f_{max}$$

Anti-aliasing The correct way to downsize an image is to first apply a **low-pass filter** (like a Gaussian blur) to remove high-frequency details that cannot be represented at the lower resolution. After blurring, the image can be safely sub-sampled.



1.4.2 Upsampling and interpolation

To increase an image's size, we must interpolate new pixels.

- **Nearest-neighbor interpolation**: repeat pixels; the simplest but leads to blocky results.
- **Bilinear interpolation**: compute each new pixel as a weighted average of the 4 nearest pixels in the original image; smoother results.
- **Bicubic interpolation**: adopt a 16-pixel neighborhood for an even smoother result.

Nearest-neighbor interpolation | Bilinear interpolation | Bicubic interpolation

Modern super-resolution

More advanced techniques use NNs to learn how to generate high-resolution details, or multi-image methods capturing slightly shifted photos and combining them to reconstruct a higher-resolution image.

1.5 Image derivatives and edge detection

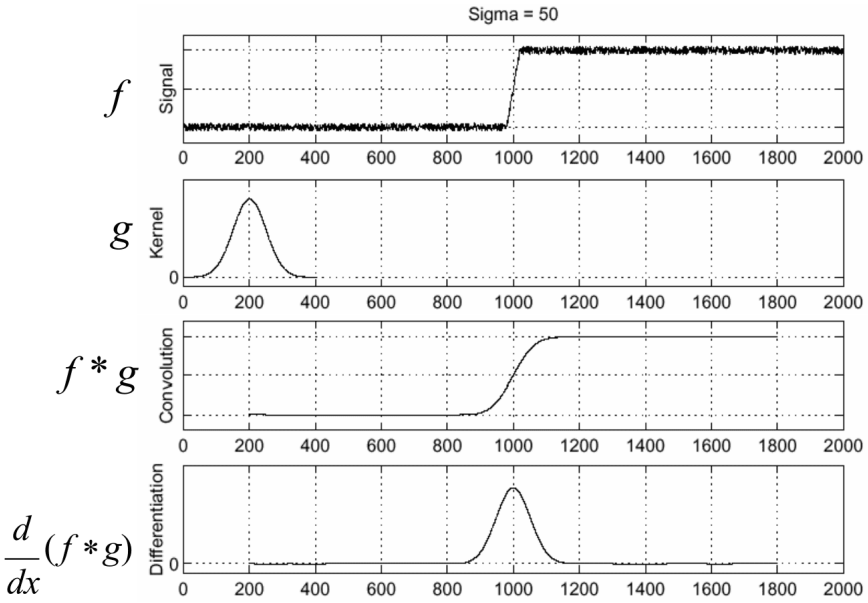
Edges are locations in an image with a **rapid change in intensity**. They are fundamental features for understanding image content.

1.5.1 The image gradient

Since an image is a 2D function $f(x, y)$, we can analyze intensity changes by computing its derivatives. These are approximated using convolution with small kernels.

- The **partial derivative** $\frac{\partial f}{\partial x}$ can be approximated by convolving with a filter like $[-1, 1]$ or $[1, -1]$ (vertical for $\frac{\partial f}{\partial y}$).
- The **gradient**, $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$, is a **vector** that points in the direction of the most rapid intensity increase.
- The **gradient magnitude**, $||\nabla f|| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$, measures the edge strength.
- The **gradient orientation**, $\theta = \text{atan2}\left(\frac{\partial f}{\partial y}, \frac{\partial f}{\partial x}\right)$, gives the direction of the edge.

Handling noise Directly applying derivative filters is highly sensitive to noise, as noise creates many small, rapid intensity changes. The solution is to first smooth the image with a Gaussian filter.

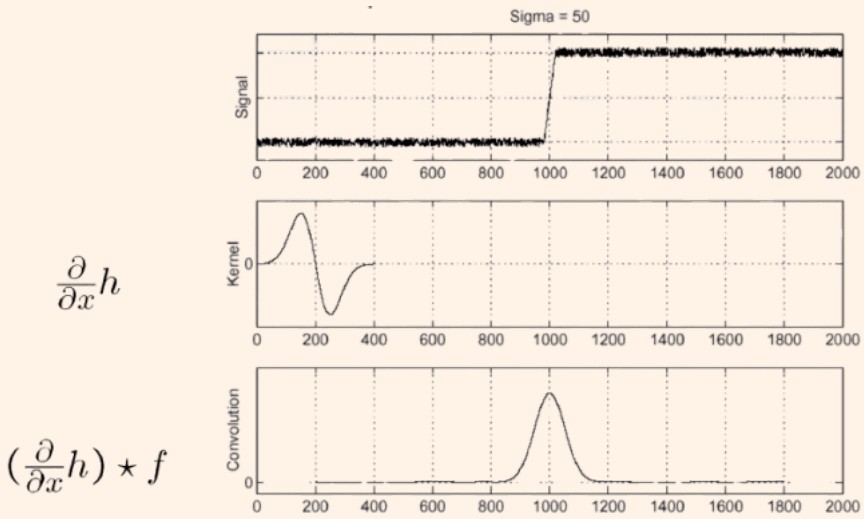


Derivative theorem of convolution

The associative property of convolution allows us to combine the smoothing and derivative steps into a single, efficient operation:

$$\frac{\partial}{\partial x}(G * f) = \left(\frac{\partial}{\partial x}G\right) * f$$

Instead of blurring the image and then taking its derivative, we can take the derivative of the Gaussian kernel itself and convolve the image with this new **Derivative of Gaussian (DoG)** filter.

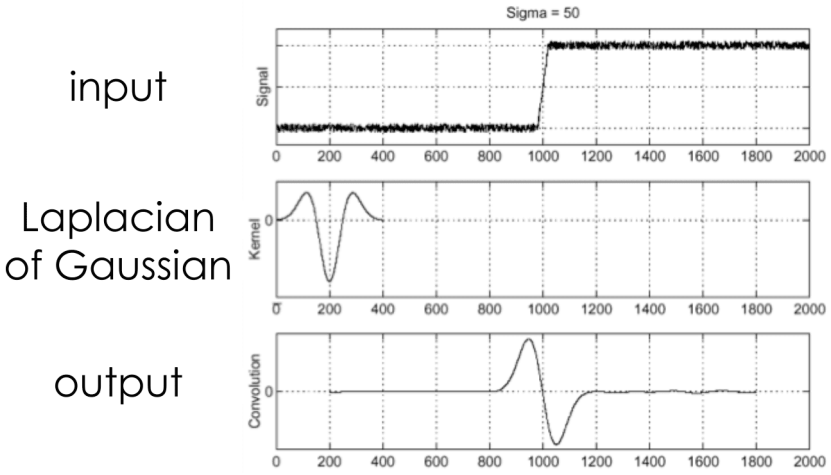


1.5.2 Second-order derivatives: the Laplacian

The second derivative can also be used to find edges. The Laplacian of an image is given by $\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$, which is a **scalar**.

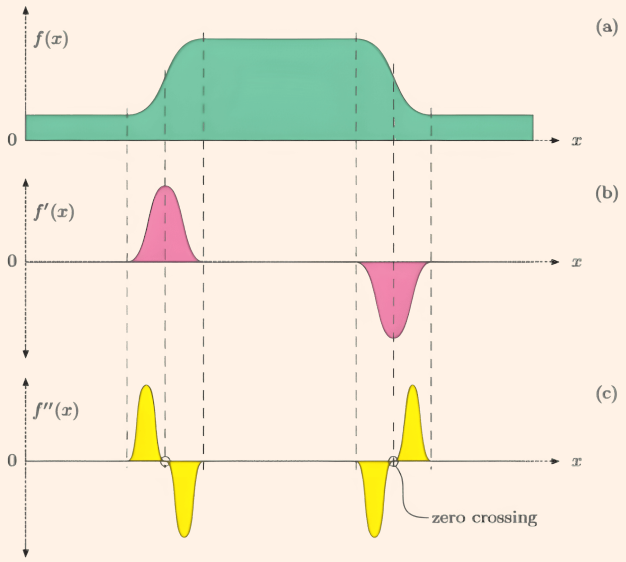
- In a 1D signal, an edge corresponds to a peak in the first derivative and a **zero-crossing** in the second derivative.
- Similar to the DoG filter, we can combine the Laplacian with Gaussian smoothing to create

a **Laplacian of Gaussian (LoG)** filter. Edges are then located by finding zero-crossings in the filtered image.

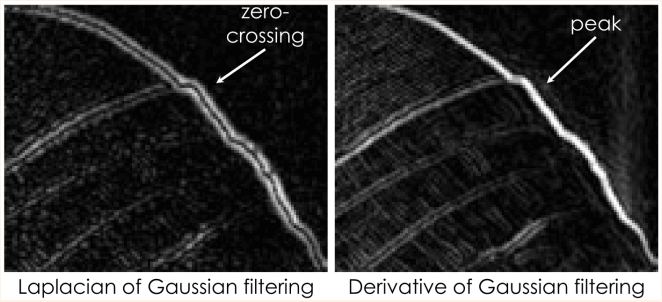


Zero-crossing of first and second derivative

The first derivative at an edge corresponds to a maximum, whereas the second derivative is zero.



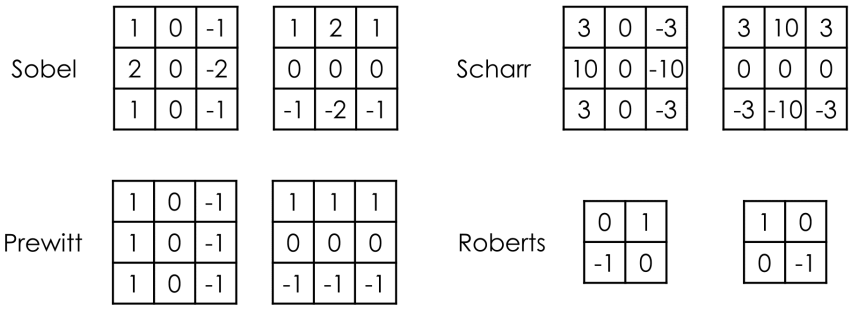
It is easier to understand when a function is zero than computing its maximum.



Zero crossings are more accurate at localizing edges but DoG is computationally more efficient.

1.5.3 The Sobel operator

The *Sobel operator* is an efficient approximation of DoG. It is a **separable filter** composed of a 1D derivative kernel and a 1D blurring kernel. For example, the horizontal Sobel filter combines ‘[1, 2, 1]‘ (blurring) and ‘[1, 0, -1]‘ (derivative). Other common derivative filters include **Prewitt**, **Roberts**, and **Scharr**.



1.5.4 The Canny edge detection pipeline

The *Canny edge detector* is a multi-stage algorithm that is considered to be the standard classical edge detection method.

1. **Filter image**: Smooth the image with a Gaussian filter and compute the gradient magnitude and orientation using a DoG filter (or an approximation like Sobel). Of course we can just directly convolve with DoG (or Sobel).

2. **Non-maximum suppression**: For each pixel, check if its magnitude is a local maximum along the direction of its gradient: if not, suppress it by setting it to 0. This produces thin, single-pixel-wide edges.

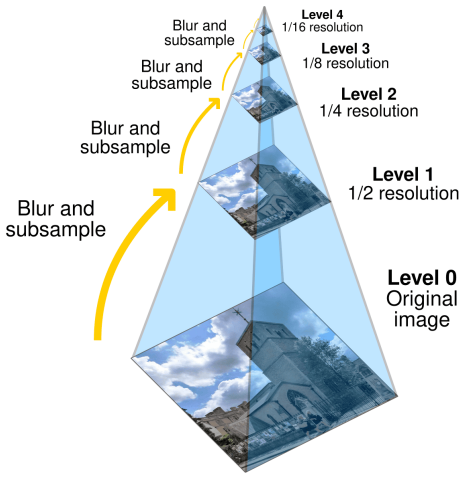
3. **Linking and hysteresis thresholding**: This step eliminates weak edges caused by noise while preserving weak edges that are part of a true contour. Two thresholds are used: a *high* threshold and a *low* threshold.

    - Pixels with gradient magnitude above the high threshold are marked as "strong" edges and are kept.
    - Pixels with magnitude between the low and high thresholds are marked as "weak" edges, and they are kept only if connected to a strong edge pixel.
    - Pixels below the low threshold are discarded.

2 Image pyramids

2.1 Gaussian pyramids

A *Gaussian pyramid* is a multi-resolution image representation where each level is created by Gaussian filtering and downsampling the previous level by a factor of 2, resulting in progressively smaller, blurrier images, useful for **coarse-to-fine search**.



As we move to higher levels of the pyramid (lower resolutions), high-frequency details are smoothed out and lost, while low-frequency components and uniform regions are preserved.

Blurring is lossy

It is not possible to reconstruct the original high-resolution image from a lower-resolution level.

Size of a Gaussian pyramid

If each level is subsampled by a factor of 2, it means both the width and the height are halved $\implies$ the next level will have 1/4 pixels of the previous level. In fact, the total number of pixels of the entire pyramid only requires about 33% more storage than the original image. This is given by the following geometric series:

$$N_{\text{total}} = N + \frac{N}{4} + \frac{N}{16} + \frac{N}{64} + \cdots = N \sum_{i=0}^{\infty} \left(\frac{1}{4}\right)^i = N \frac{1}{1 - 1/4} = \frac{4}{3} N$$

2.2 Laplacian pyramids

The *Laplacian pyramid* is a **lossless representation** that stores the high-frequency details lost between two levels of a Gaussian pyramid by computing a *residual image*.

Let's define two key operations based on the construction process:

- **REDUCE(I)**: Create the next level G_{i+1} of the Gaussian pyramid by blurring and then downsampling G_i.

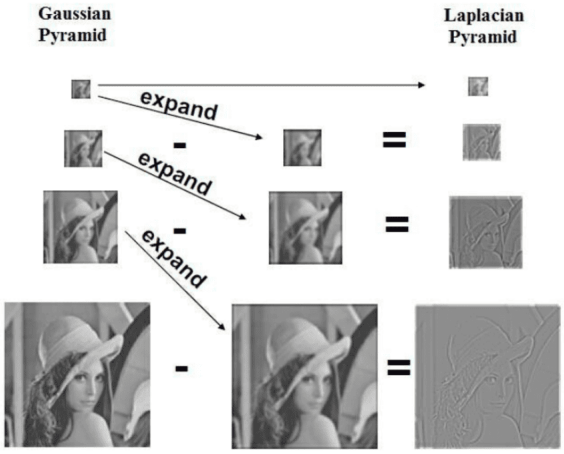
$$G_{i+1} = \text{REDUCE}(G_i) = \text{downsample}(\text{blur}(G_i))$$

- **EXPAND(I)**: Predict a higher-resolution image G_i from a lower-resolution G_{i+1} by upsampling and then blurring (to smooth artifacts from upsampling).

$$\text{EXPAND}(G_{i+1}) = \text{blur}(\text{upsample}(G_{i+1}))$$

The *residual* L_i, which forms a **level of the Laplacian pyramid**, is the difference between G_i and the expanded version of G_{i+1}: we are subtracting the prediction made from the lower-resolution image from the actual higher-resolution image.

$$L_i = G_i - \text{EXPAND}(G_{i+1})$$



The complete Laplacian pyramid representation consists of the set of all these residual images $\{L_0, L_1, \ldots, L_{n-1}\}$ and the final, smallest Gaussian level G_n.

2.2.1 Reconstruction

The original image can be perfectly reconstructed from the Laplacian pyramid by reversing the process. Starting with the smallest image G_n, we repeatedly upsample it and add the corresponding residual from the Laplacian pyramid. The reconstruction algorithm is:

1. Start with the top-level image, G_n.
2. For each level i from $n-1$ down to 0:

$$G_i = \text{EXPAND}(G_{i+1}) + L_i$$

3. The final result, G_0, is the perfectly reconstructed original image.

Why "Laplacian"? Residual images are termed "Laplacian" because their creation (image minus blurred version, a Difference of Gaussians) approximates the LoG operator.

$$\underbrace{G_i - \text{EXPAND}(G_{i+1})}_{\text{Difference of Gaussians}} \approx \underbrace{\nabla^2 G}_{\text{Laplacian of Gaussian}}$$

2.2.2 Summary

$$\boxed{\begin{aligned} G_{i+1} &= \text{REDUCE}(G_i) \\ L_i &= G_i - \text{EXPAND}(G_{i+1}) \\ G_i &= L_i + \text{EXPAND}(G_{i+1}) \end{aligned}}$$

2.3 Applications of pyramids

Image pyramids are a fundamental tool used in many areas of image processing and computer vision, applied in image compression, denoising, multi-scale registration (aligning images from coarse to fine), and multi-scale detection (finding objects at various scales).

2.3.1 Image blending

Laplacian pyramids provide a powerful technique for seamlessly blending two images together, avoiding the sharp seam that would result from a simple cut-and-paste.

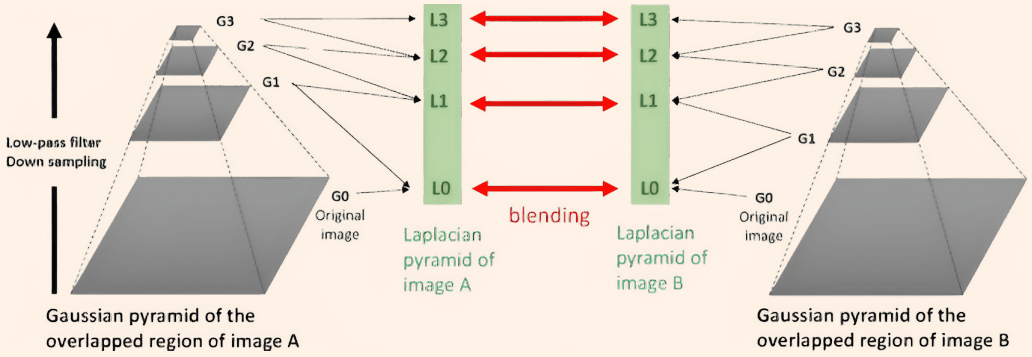
Pyramid blending algorithm

To blend two images A and B using a mask image M:

1. Build Laplacian pyramids for the two source images, LA and LB.
2. Build a Gaussian pyramid for the mask, GM. The mask values (from 0 to 1) will serve as blending weights at each scale.
3. For each level j, create a combined Laplacian level LC_j by taking a weighted average of the corresponding levels from LA and LB, using the mask pyramid GM_j as weights:

$$LC_j(x,y) = GM_j(x,y) \cdot LA_j(x,y) + (1 - GM_j(x,y)) \cdot LB_j(x,y)$$

4. Collapse the newly created combined Laplacian pyramid LC to obtain the final blended image.



3 Frequency domain

3.1 The frequency domain and Fourier series

Any univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies, under some conditions.

3.1.1 Building signals from sinusoids

The basic building block for Fourier analysis is the sinusoid:

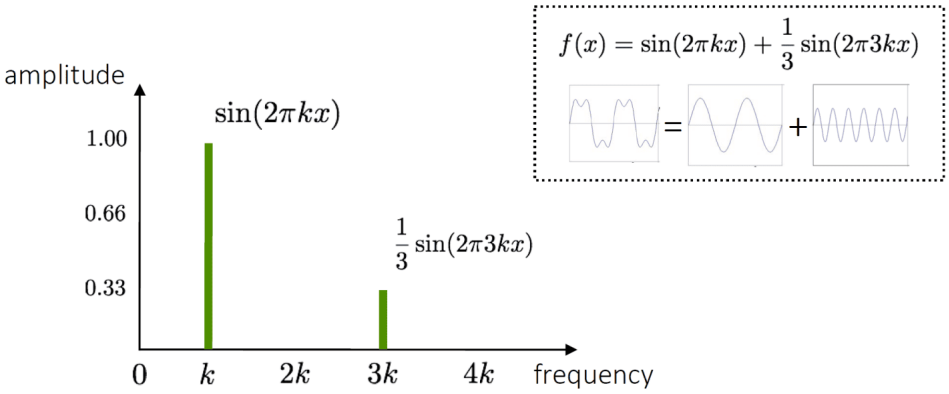
$$A\sin(\omega x + \phi)$$

This function is defined by three key parameters:

- **Amplitude (A)**: The strength or intensity of the wave.
- **Angular Frequency (ω)**: Determines how fast the wave oscillates. The period is $T = 2\pi/|\omega|$.
- **Phase (ϕ)**: The starting position or offset of the sinusoid.

Fourier's claim is that by **summing enough of these sinusoids** with different frequencies and amplitudes, we can **approximate any periodic signal**, such as a square wave.

Frequency spectrum A plot that shows the amplitude of each frequency component present in the signal. For a signal $f(x) = \sin(2\pi kx) + \frac{1}{3}\sin(2\pi 3kx)$, the spectrum would show a large spike at frequency k and a smaller spike (one-third the amplitude) at frequency $3k$. The value at frequency 0 represents the signal's average value (the DC component).



3.2 The Fourier transform

While Fourier series represent periodic functions, the *Fourier transform* extends this idea to **non-periodic functions** by representing them as an integral over a continuous spectrum of frequencies.

From spatial to frequency domain The Fourier transform converts a signal $f(x)$ from the spatial domain to a function $F(\omega)$ in the frequency domain. The function $F(\omega)$ is complex-valued, meaning it holds both the amplitude and phase for each frequency ω.

$$F(\omega) = R(\omega) + iI(\omega)$$

The amplitude A and phase ϕ can be recovered from the real (R) and imaginary (I) parts:

$$A = \sqrt{R(\omega)^2 + I(\omega)^2} \qquad \phi = \tan^{-1}\left(\frac{I(\omega)}{R(\omega)}\right)$$

This transformation is invertible, meaning we can always go back from the frequency domain to the spatial domain.

3.2.1 Mathematical definition

The continuous Fourier transform and its inverse are formally defined using complex exponentials.

Forward Transform (Spatial → Frequency)

$$F(\omega) = \int_{-\infty}^{+\infty} f(x) e^{-i\omega x} dx$$

Inverse Transform (Frequency → Spatial)

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} F(\omega) e^{i\omega x} d\omega$$

The connection to sinusoids is given by **Euler's formula**:

$$e^{i\theta} = \cos(\theta) + i\sin(\theta)$$

This shows that the transform is essentially measuring how much of a given sine and cosine frequency is present in the signal $f(x)$.

3.2.2 The discrete fourier transform (DFT)

For digital signals and images, we use the *Discrete Fourier Transform* (**DFT**). For a 1D signal $f(x)$ of length N, the DFT is:

$$F(k) = \sum_{x=0}^{N-1} f(x) e^{-j2\pi kx/N}$$

This can be expressed as a matrix multiplication $\mathbf{F} = \mathbf{Wf}$.

The Fast Fourier Transform (FFT)

Directly computing the DFT via matrix multiplication has a complexity of $O(N^2)$. In practice, a highly efficient algorithm called the **Fast Fourier Transform (FFT)** is used, which computes the DFT in $O(N \log N)$ time, making it practical for image processing.

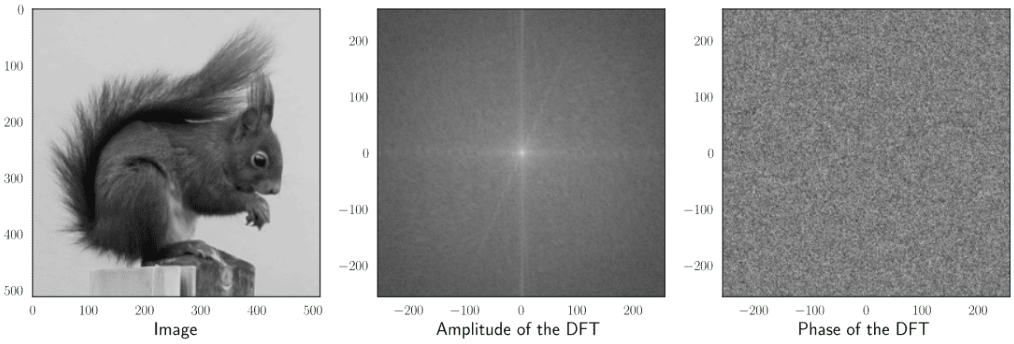
3.3 Fourier analysis in 2D

The Fourier transform can be extended to 2D to analyze images. An image $f(x, y)$ is transformed into its 2D frequency representation $F(u, v)$, where u and v are horizontal and vertical frequencies, respectively.

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

The resulting 2D Fourier spectrum is typically visualized with the zero frequency (DC component) at the center.

- **Low frequencies** are near the center and correspond to large-scale, smooth structures in the image.
- **High frequencies** are far from the center and correspond to fine details, edges, and texture.
- The orientation of lines in the spectrum corresponds to the orientation of edges in the image. For example, vertical edges create horizontal lines in the frequency domain.



The importance of phase

The DFT decomposes an image into its frequency components, where each component has an **amplitude** and a **phase**.

- The DFT amplitude represents how much of a particular frequency is present in the image $\implies$ it lacks spatial information.
- The DFT phase represents the spatial arrangement of each frequency component $\implies$ it defines the location of edges, lines, corners, and textures.

If we reconstruct an image using only phasen the resulting image will be **recognizable** but distorted, with strange intensity variations.

3.4 Filtering in the frequency domain

One of the most powerful applications of the Fourier transform is for filtering.

3.4.1 The convolution theorem

The convolution theorem states that convolution in the spatial domain is equivalent to element-wise multiplication in the frequency domain.

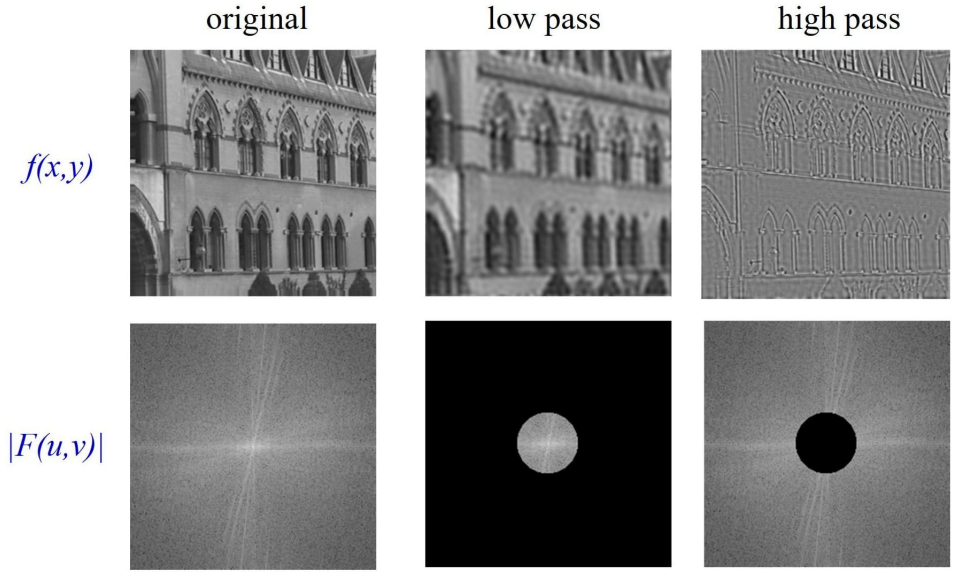
$$\mathcal{F}\{g * h\} = \mathcal{F}\{g\} \cdot \mathcal{F}\{h\}$$

This means that instead of performing a computationally expensive convolution in the spatial domain, we can:

1. Transform the image and the kernel to the frequency domain using FFT.
2. Multiply them together element-wise.
3. Transform the result back to the spatial domain using an inverse FFT.

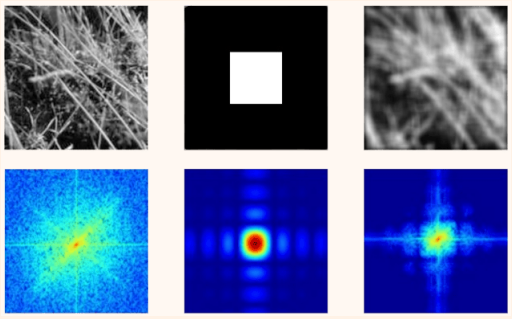
Filtering examples This principle allows for intuitive filtering operations by manipulating the frequency spectrum directly:

- **Low-pass filtering**: To blur an image, we multiply its spectrum by a mask that preserves low frequencies (near the center) and attenuates high frequencies. A Gaussian function is an excellent low-pass filter.
- **High-pass filtering**: To sharpen an image or detect edges, we use a mask that attenuates low frequencies and preserves high frequencies.
- **Band-pass filtering**: To isolate features of a specific scale, we can use a mask that preserves a ring of frequencies.

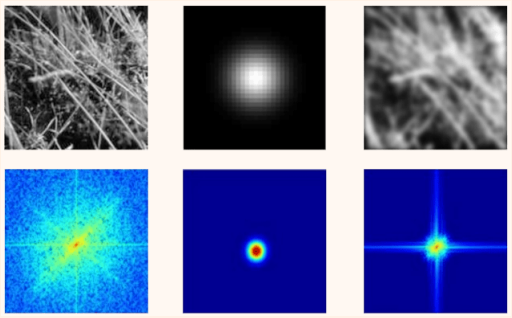


Box filter vs. Gaussian filter

A **box filter** is abrupt in the spatial domain. Its Fourier transform, a **sinc function**, exhibits significant sidelobes. These non-ideal frequency characteristics result in an imperfect frequency cutoff and can cause **ringing artifacts** in the blurred spatial image.



In contrast, a **Gaussian filter** is smooth in the spatial domain. Its Fourier transform is also a smooth Gaussian, ensuring a clean frequency attenuation.



3.5 Sampling

The *Nyquist-Shannon sampling theorem* states that a continuous signal can be perfectly reconstructed from its discrete samples **without aliasing** if and only if

$$f_s \geq 2f_{\max}$$

where the minimum sampling rate $2f_{\max}$ is called the *Nyquist frequency*.

Application to Gaussian pyramids

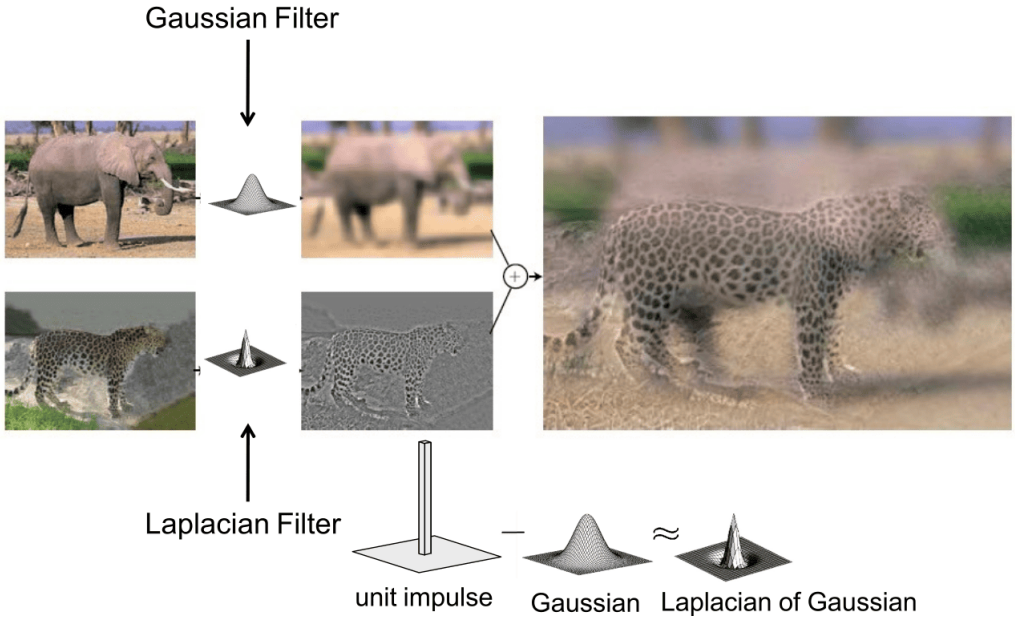
Gaussian blurring (low-pass filtering) reduces $f_{\max}$, thus preventing aliasing when the image is subsequently downsampled.

3.6 Hybrid images

Hybrid images are created by combining the low-frequency components of one image with the high-frequency components of another.

1. A **low-pass filter** (e.g., Gaussian) is applied to the first image, retaining only its coarse structure.
2. A **high-pass filter** (e.g., Laplacian) is applied to the second image, retaining only its fine details and edges.
3. The two resulting images are added together.

The final hybrid image is perceived differently depending on the viewing distance. From far away, our visual system picks up the low-frequency image; up close, the high-frequency details become dominant.

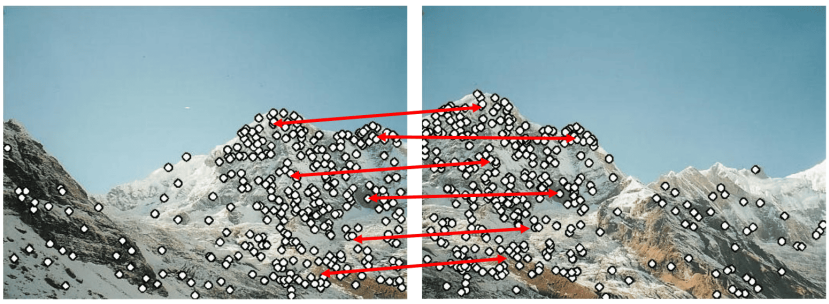


4 Local features

4.1 Introduction to local features

A *feature* is a piece of information about the content of an image. We can distinguish between two main types:

- **Global features** describe an image as a whole to represent an entire object. They are used for object deteciton (is the object present, yes/no?).
- **Local features** describe image patches or keypoints within an object. They are used for object recognition, where matching parts of objects (local features, indeed) is necessary.

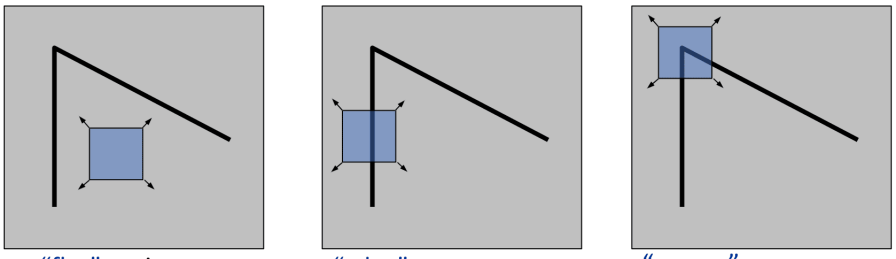


Properties of good local features

Good local features must be **numerous** enough for matching and **distinctive** for unambiguous correspondence. Crucially, they must be **invariant** to photometric and geometric transformations.

4.2 Harris corner detector

The *Harris corner detector* identifies important points by shifting a small window around them and check where strong appearance changes occur.

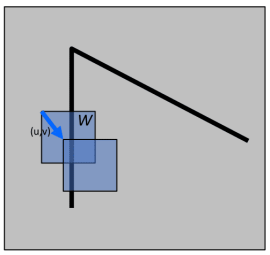


4.2.1 The mathematics of corner detection

To quantify the intensity change, we compute the *sum of squared differences* (**SSD**) for a window W shifted by a small offset (u, v).

$$E(u,v) = \sum_{(x,y)\in W} [I(x+u, y+v) - I(x,y)]^2$$

A corner is a point where this error $E(u, v)$ is high for all small shifts (u, v).



However, computing this directly for each pixel is too expensive.

Small motion approximation For a small shift, we can approximate the intensity change using a first-order Taylor series expansion:

$$I(x+u, y+v) \approx I(x,y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v = I(x,y) + I_x u + I_y v$$

Substituting this into the SSD formula gives:

$$E(u,v) \approx \sum_{(x,y)\in W} [I_x u + I_y v]^2$$

This can be expanded into a quadratic form:

$$E(u,v) \approx Au^2 + 2Buv + Cv^2$$

where $A = \sum I_x^2$, $B = \sum I_x I_y$, and $C = \sum I_y^2$, with the sums taken over the window W.

The second moment matrix The quadratic form can be expressed in matrix notation:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\begin{bmatrix} A & B \\ B & C \end{bmatrix}}_{\mathbf{H}} \begin{bmatrix} u \\ v \end{bmatrix}$$

The matrix $\mathbf{H}$ is the **second moment matrix** (also called the structure tensor). It summarizes the gradient distribution within the window W.

$$\mathbf{H} = \sum_{(x,y)\in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

4.2.2 Interpreting the second moment matrix

The shape of the quadratic error surface $E(u,v)$ is determined by the eigenvalues, λ_1 and λ_2, of the matrix $\mathbf{H}$.

- The eigenvectors of $\mathbf{H}$ correspond to the directions of maximum and minimum intensity change.
- The eigenvalues $\lambda_{\max}$ and $\lambda_{\min}$ represent the magnitude of this change in those directions.

This allows us to classify the region based on the eigenvalues:

- **Flat**: Both λ_1 and λ_2 are small. The error is low in all directions.
- **Edge**: One eigenvalue is large and the other is small (e.g., $\lambda_1 \gg \lambda_2$). The error is high in one direction but low along the edge.
- **Corner**: Both λ_1 and λ_2 are large and have similar magnitude. The error is high in all directions.

4.2.3 The harris operator and algorithm

Computing eigenvalues for every pixel is inefficient. The Harris detector uses a **response function** R that approximates the eigenvalue behavior without explicitly calculating them.

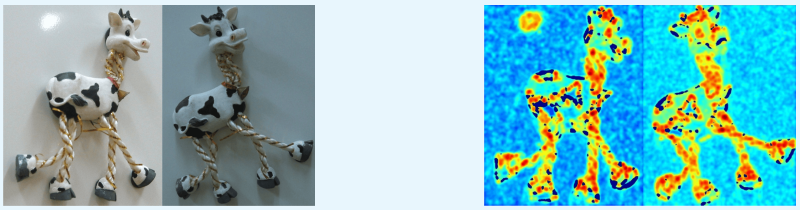
$$R = \det(\mathbf{H}) - k\text{trace}^2(\mathbf{H}) = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$$

Here, k is an empirical constant (typically 0.04-0.06). The value of R indicates the "**cornerness**": it is large and positive for a corner, large and negative for an edge, and small for a flat region.

The Harris detector algorithm

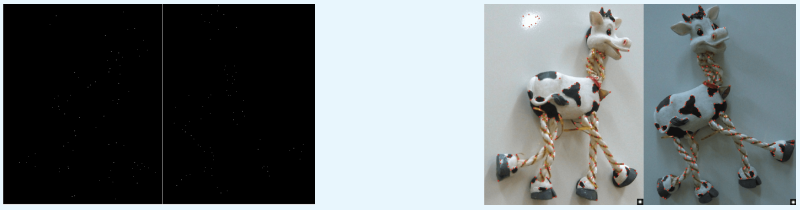
The complete pipeline is as follows:

1. Compute image derivatives I_x and I_y (e.g., using Sobel filters).
2. For each pixel, compute the second moment matrix $\mathbf{H}$ over a Gaussian window around it.
3. Compute the Harris corner response R for each pixel.



4. Threshold the response map R to identify strong corner candidates.

5. Perform non-maximum suppression to find the final corner points.

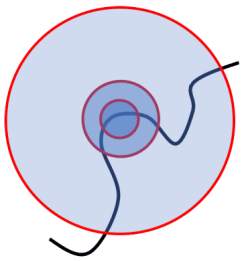


4.2.4 Properties and limitations

The Harris detector is **equivariant** to rotation (its second moment ellipse rotates, but shape/eigenvalues are preserved) and translation. It is fully **inviariant** to brightness shifts ($+b$) but only partially to constrast changes ($\times a$), as its response R scales, affecting thresholding. However, it is **not scale invariant**, because a corner can appear flat or as an edge when zoomed in.

4.3 Scale-invariant blob detection

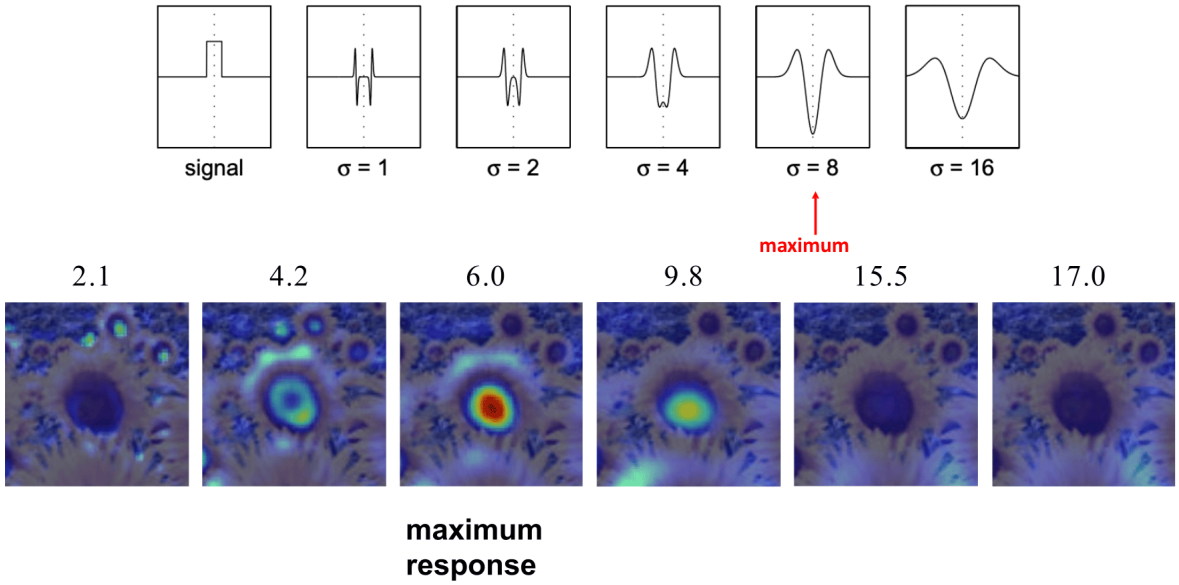
To overcome Harris's scale limitation, scale-invariant *blobs* are detected by finding their **characteristic scale**: the scale σ in a 3D **scale-space** (x, y, σ) where a response function is maximized.



The **scale-normalized LoG** is a common blob detector, defined as:

$$\nabla^2_{\text{norm}} g = \sigma^2 \left(\frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} \right)$$

Maximizing this LoG response across scales yields the characteristic scale.



Implementation

Instead of applying increasingly larger filters to the image, it is more efficient to build a Gaussian pyramid and apply a fixed-size filter to each level.

1. Build a scale-space by computing the LoG response for a range of scales σ.
2. Find local maxima in the 3D position-scale space (x, y, σ).
3. The location (x, y) and characteristic scale σ of each maximum define a detected feature.

Fixed-size LoG filter and effective scale

When using a Gaussian pyramid for scale-invariant blob detection, a "fixed-size LoG filter" refers to a filter kernel with fixed spatial dimension and internal σ. Applying this fixed filter to different pyramid levels effectively simulates varying scales:

- **Level 0 (original image):** The filter detects features at an effective scale roughly proportional to σ_{filter}.
- **Level 1 (downsampled by 2x):** The image is now half the resolution, and each pixel of the same filter now covers 2 $times$2 pixels from the original image. Therefore, the effective scale relative to the original image becomes $2 \times \sigma_{\text{filter}}$.
- **Level 2 (downsampled by 4x):** Similarly, the effective scale becomes $4 \times \sigma_{\text{filter}}$.

The characteristic scale for a detected blob is the scale of the pyramid level at which its LoG response is maximized.

5 Local descriptors

5.1 Designing feature descriptors

A *feature descriptor* is a **vector representation** of the patch surrounding a keypoint. The goal is to design descriptors such that **patches with similar content have similar descriptors**, enabling robust matching.

Robustness

A good descriptor must be robust to photometric and geometric transformations.

5.1.1 Evolution of a descriptor

We can build up the intuition for a robust descriptor by starting with simple ideas and addressing their shortcomings.

1. **Raw image patch**: pixel intensities; perfect for exact matches but highly sensitive to brightness/position changes.
2. **Image gradients**: intensity variations; invariant to brightness shifts but sensitive to deformation/rotation and contrast changes $\implies$ this is why normalization in HOG/SIFT is applied.
3. **Color histogram**: color distribution; invariant to rotation/scale but loses all spatial layout, making it non-discriminative.
4. **Spatial histograms**: histograms per cell (like in HOG or the regions in SIFT).
   - **Partially invariant to deformation**: by dividing the image into spatial bins, they allow for minor shifts/deformations within them without drastically changing the descriptor.
   - **Sensitive to rotation**: the spatial bins are fixed relative to the image frame, so a rotated object will cause its features (gradients) to fall into different spatial bins, significantly altering the descriptor unless a dominant orientation is first established and used for alignment (like SIFT).
5. **Orientation normalization**: achieves full rotation invariance by aligning the patch to its dominant gradient orientation before descriptor computation, decoupling the descriptor from the patch's original angle.

5.1.2 Comparing histograms

Measures for comparing histogram-based descriptors are:

- **Histogram intersection**: Measures the common part of two histograms. It is robust.
$$\cap(Q, V) = \sum_i \min(q_i, v_i)$$
- **Euclidean distance**: A simple L2-norm distance. Not very robust as it weighs all bins equally.
$$d(Q, V) = \sqrt{\sum_i (q_i - v_i)^2}$$
- **Chi-square distance**: More discriminative because it weighs bins differently.
$$\chi^2(Q, V) = \sum_i \frac{(q_i - v_i)^2}{q_i + v_i}$$

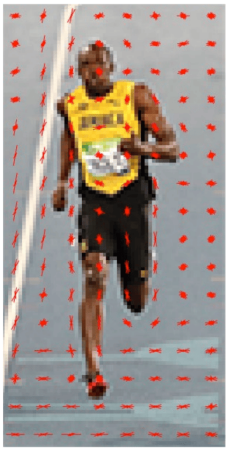
5.2 Histogram of Oriented Gradients (HOG) descriptor

HOG is a powerful feature descriptor used for human detection, which combines the ideas of using gradients, spatial cells, and local contrast normalization.

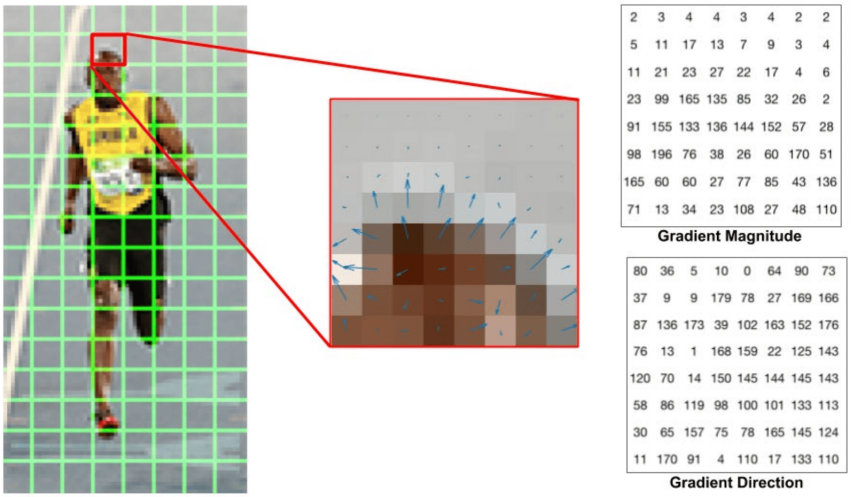
5.2.1 HOG pipeline

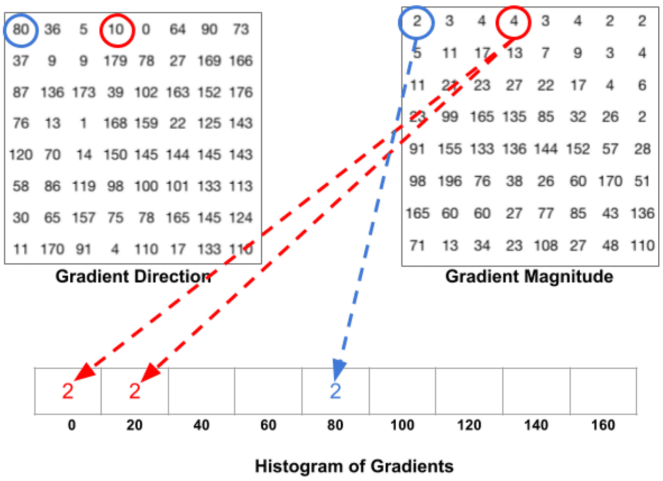
The descriptor is computed for a fixed-size image patch (e.g., 64x128 pixels for human detection).

1. **Gradient computation**: First, calculate the horizontal and vertical gradients of the patch to get the magnitude and direction for each pixel, using **unsigned** gradients with angles from 0-180° (so opposite directions are treated the same).



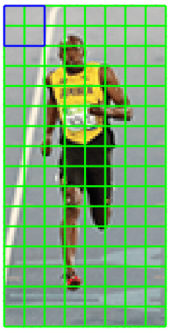
2. **Cell histograms**: Divide the patch into small spatial regions called **cells** (e.g., 8x8 pixels). For each cell, create a histogram of gradient orientations with 9 bins (for angles 0°, 20°, 40°, ... 160°). Each pixel in the cell casts a weighted vote into the histogram: the vote's bin is determined by the pixel's gradient orientation, and the vote's value is determined by its gradient magnitude. To avoid aliasing, votes are interpolated bilinearly between adjacent bins.





3. **Block normalization**: To achieve robustness to contrast changes, group adjacent cells into larger regions called **blocks** (e.g., 2x2 cells, resulting in a 16x16 pixel block). Concatenate the histograms of all cells within a block into a single vector. Normalize this block vector to make the descriptor **robust to local photometric changes**.

$$v_{\text{norm}} = \frac{v}{\sqrt{||v||_2^2 + \epsilon}}$$



4. **Feature vector**: The normalized block vectors are concatenated to form the final HOG descriptor. Blocks are overlapping (e.g., a window is moved by 8 pixels at a time), meaning interior cells contribute to multiple block vectors, providing a redundant and robust representation.

HOG for pedestrian detection

For a 64x128 patch with 8x8 cells and 2x2 cell blocks with a step size of 8 pixels, the final HOG descriptor is a 3780-dimensional vector. This high-dimensional vector can be fed into a linear classifier like an SVM to perform tasks like pedestrian detection. The visualization of the HOG descriptor clearly captures the shape of the person, especially around the torso and legs.

HOG properties

The standard HOG descriptor is designed for a single scale and does not have a dominant orientation, making it suitable for describing objects with a canonical pose (like upright pedestrians) $\implies$ it is **not invariant to rotation**.

6 Scale-Invariant Feature Transform (SIFT)

The *Scale-Invariant Feature Transform* (**SIFT**) is a landmark algorithm for **detecting** and **describing** local features in images, designed to be **robust** to changes in scale, rotation, and illumination.

6.1 SIFT Detector

The SIFT detector identifies keypoints by finding extrema in a scale-space representation of the image.

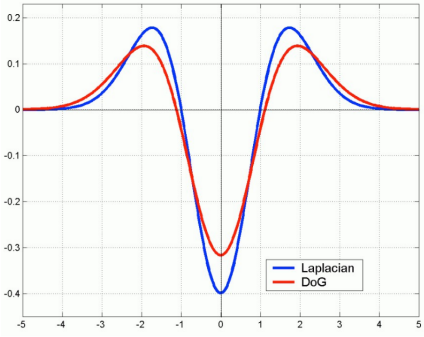
6.1.1 Scale-space construction

SIFT uses a *Difference of Gaussians* (**DoG**) pyramid, an efficient approximation of the Laplacian of Gaussian (LoG), to find features across various scales.

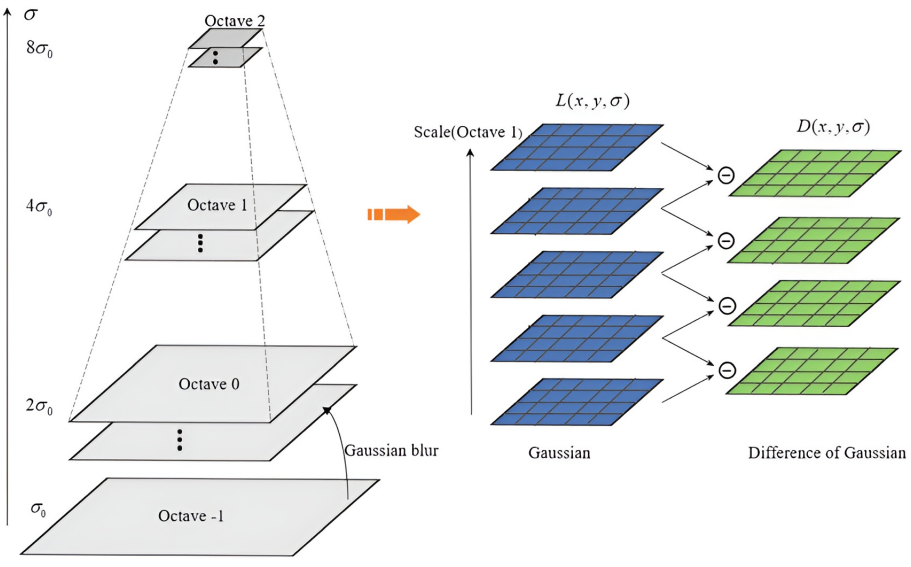
- **Approximation**: DoG approximates LoG (a powerful blob detector) by subtracting two Gaussian-blurred images at slightly different scales:

$$\mathrm{DoG}(x, y, \sigma) = G(x, y, k\sigma) - G(x, y, \sigma)$$

  Both DoG and LoG are rotation-equivariant.

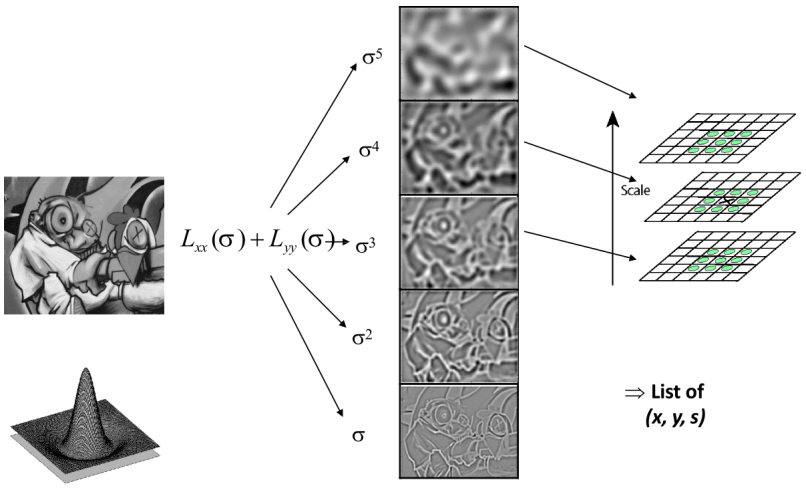


- **Pyramid structure**: A Gaussian pyramid is first built in "octaves." Each octave contains images of the same resolution, progressively blurred. To transition to the next octave, we subsample the image by a factor of 2, effectively doubling the scale. DoG images are then computed by differencing adjacent blurred images within each octave.



6.1.2 Scale-space extrema detection

A pixel is identified as a potential keypoint (in its characteristic scale) if it's a local extremum (maximum or minimum) compared to its 8 spatial neighbors in the same DoG image, and its 9 neighbors in the DoG images directly above and below it in scale-space, for a total of 26 neighbors.

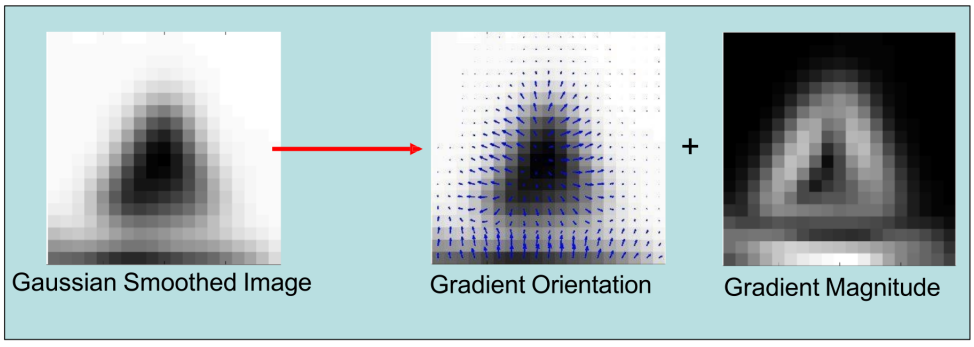


Keypoints with **low contrast**, namely a low DoG response, are then discarded as they are unstable. Furthermore, if the DoG response is on a good keypoint (a blob or a corner), it changes significantly in all directions around it, so it's distinct and well-defined; on the other hand, on an **edge** the DoG response changes strongly in one direction (across the edge) $\implies$ they're bad for matching because not distinct.

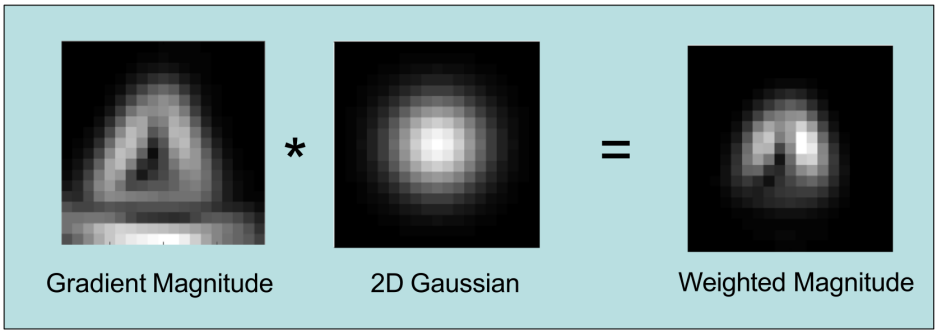
6.1.3 Orientation assignment

To make the final descriptor rotation-invariant, a **canonical orientation** is assigned to each keypoint.

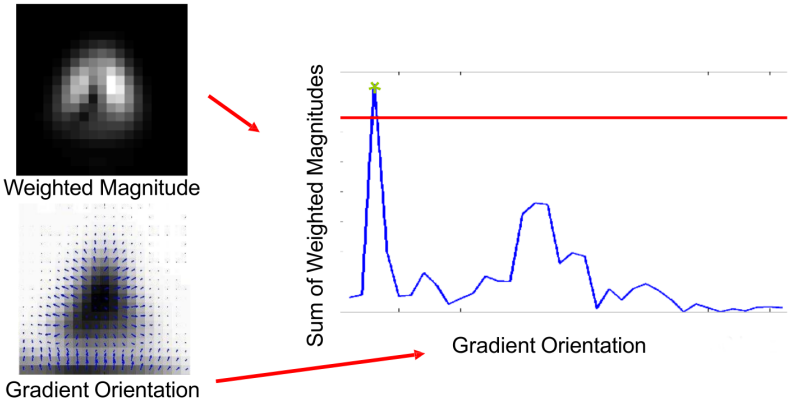
1. **Neighborhood gradient**: compute gradient magnitude and orientation for pixels in a neighborhood around the keypoint in its corresponding Gaussian image.



2. **Orientation histogram**: create a 36-bin orientation histogram where each pixel votes for its gradient orientation, weighted by its magnitude and a Gaussian window centered on the keypoint.



3. **Dominant orientation(s)**: the highest peak in the histogram defines the keypoint's primary orientation. Any other peaks above 80% of the main peak also become new keypoints (at the same location and scale) with their respective orientations.

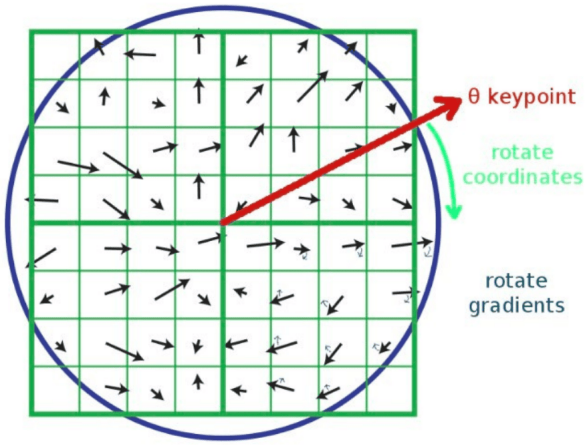


6.2 SIFT descriptor

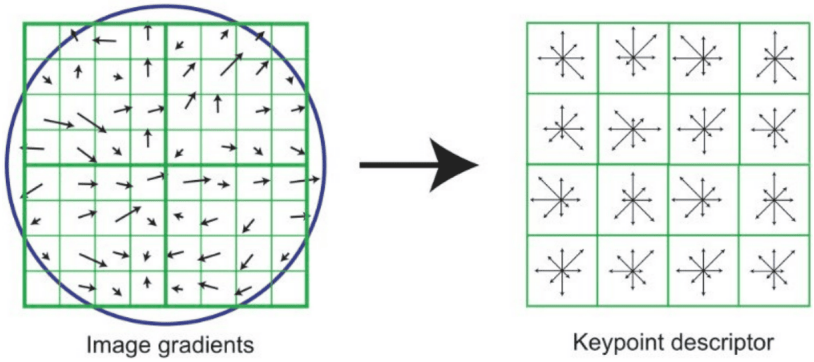
Once a keypoint has been assigned a location (x, y), a scale σ, and an orientation θ, we know we need to compute a **descriptor vector** to represent its appearance.

1. Take a 16x16 window from the Gaussian-blurred image around the keypoint in its characteristic scale.

2. Rotate the coordinate system and the gradients within this window according to the keypoint's canonical orientation. This step ensures **rotation invariance**.



3. Divide the 16x16 window into a 4x4 grid of cells (each cell is 4x4 pixels, 16 cells). For each cell, compute an **8-bin orientation histogram** (same modalities as SIFT detector).



4. **Concatenate** all 16 histograms into a single 128-dimensional vector (16 cells × 8 bins/-cell).
5. **Normalize** this 128-dim vector to unit length to gain **robustness against illumination changes**.

SIFT properties

The SIFT descriptor is **invariant** to:

- **Scale** due to detection in a scale-space pyramid.
- **Rotation** due to the orientation normalization step.
- **Illumination** due to normalization of the final descriptor vector.

It is also robust to affine transformations and noise.

6.3 Feature matching

Given a keypoint and its descriptor from a source image I_1, the goal is to find the **corresponding keypoint** in a target image I_2.

6.3.1 Lowe's ratio test

A simple approach is to find the feature in I_2 with the minimum Euclidean distance to the source descriptor. However, this can lead to many false matches for ambiguous features. A more robust method is the **nearest neighbor distance ratio test** (*Lowe's ratio test*).

1. For a feature f_1 in image I_1, find its two nearest neighbors in I_2, f_2 (the best match) and f_2' (the second-best match).
2. Compute the ratio of their distances:

$$\text{ratio} = \frac{||f_1 - f_2||}{||f_1 - f_2'||}$$

3. If this ratio is below a certain threshold (e.g., 0.75 or 0.8), the match is accepted. Otherwise, it is rejected as ambiguous.

6.4 Evaluating matching performance

To objectively measure the performance of a feature matcher, we use evaluation metrics that rely on ground-truth data (an "oracle" that tells us which matches are correct).

True/False Positives and ROC Curves By setting a threshold on the matching distance (or distance ratio), we can classify each potential match as accepted or rejected.

- **True Positive (TP)**: An accepted match that is correct.
- **False Positive (FP)**: An accepted match that is incorrect.

By varying the threshold, we can plot the **True Positive Rate (Recall)** against the **False Positive Rate**. This is known as a **Receiver Operating Characteristic (ROC) curve**.

$$\text{True Positive Rate (Recall)} = \frac{\#\text{TP}}{\#\text{Total Correct Matches}}$$

$$\text{False Positive Rate} = \frac{\#\text{FP}}{\#\text{Total Incorrect Matches}}$$

A single metric, the **Area Under the Curve (AUC)**, summarizes the overall performance. An AUC of 1.0 is a perfect classifier, while 0.5 is random guessing.

On "accuracy"

Simple accuracy, $(TP+TN)/(TP+TN+FP+FN)$, is a poor metric for feature matching because the number of true negatives (correctly rejected non-matches) is astronomically large and dominates the calculation. Precision and Recall are more informative.

- **Precision**: Of all the matches we accepted, what fraction was correct? $\frac{TP}{TP+FP}$
- **Recall**: Of all the correct matches that exist, what fraction did we find? $\frac{TP}{TP+FN}$
- **F1-score**: The harmonic mean of precision and recall, providing a single balanced measure.

6.5 SURF: Speeded-Up Robust Features

SURF is a faster alternative to SIFT with comparable quality.

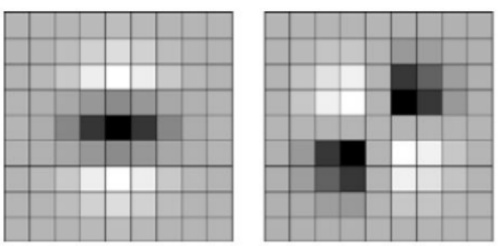
6.5.1 Hessian-based interest point detection

Instead of using DoG, SURF detects blobs at locations where $\det(\mathbf{H})$ is maximal, where $\mathbf{H}$ at a point $\mathbf{x}$ and scale σ is:

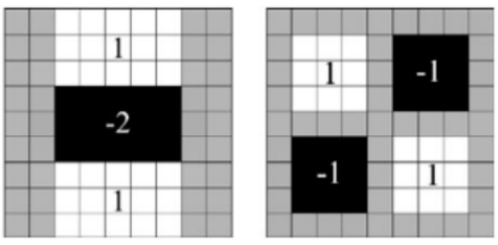
$$\mathbf{H}(\mathbf{x}, \sigma) = \begin{bmatrix} L_{xx}(\mathbf{x}, \sigma) & L_{xy}(\mathbf{x}, \sigma) \\ L_{xy}(\mathbf{x}, \sigma) & L_{yy}(\mathbf{x}, \sigma) \end{bmatrix}$$

where L_{xx} is the convolution of the image with the second partial derivative of a Gaussian $\frac{\partial^2 G}{\partial x^2}$.

Hessian approximation with integral images Direct computation of $\mathbf{H}$ is slow, so SURF approximates the Gaussian second-order derivatives with simple **box filters**.

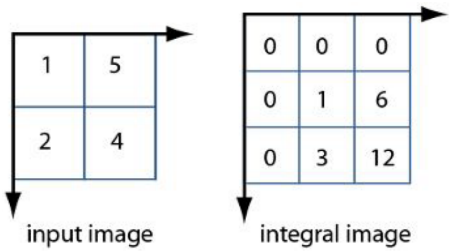


The Gaussian second order partial derivative in y- and xy-direction.



Box filter approximations of the Gaussian second order partial derivatives.

The convolution with these box filters can be computed extremely quickly using *integral images*. An integral image I_Σ at a location $\mathbf{x} = (x, y)$ stores the sum of all pixel intensities in the rectangle from the origin to $\mathbf{x}$.



The approximate determinant of the Hessian is then:

$$\det(H_{approx}) = D_{xx}D_{yy} - (wD_{xy})^2$$

where D_{xx}, D_{yy}, D_{xy} are the responses of the box filters and $w = 0.9$ is a weighting factor.

6.5.2 SURF descriptor

1. **Orientation assignment:** Compute *Haar wavelet* responses in the x and y directions within a circular neighborhood around the interest point. Weight them by a Gaussian and use them to find a dominant orientation for **rotation invariance**.
2. **Feature vector:** A square window of size $20s \times 20s$ (where s is the detection scale) is aligned with the dominant orientation and divided into a 4×4 grid of cells = 16 cells. For each cell, the sum of Haar wavelet responses $(\sum dx, \sum dy)$ and their absolute values $(\sum |dx|, \sum |dy|)$ are computed. Concatenating these 4 values for all 16 cells creates a **64-dimensional descriptor vector**.

6.6 Binary feature descriptors

Less expensive alternative to SIFT that generate **small binary strings** that are very fast to compute and compare.

6.6.1 General approach

1. Select an image patch around a detected keypoint.
2. Select a set of pixel pairs (s_1, s_2) within that patch.
3. For each pair, perform a simple intensity comparison to generate one bit:

$$b = \begin{cases} 1 & \text{if } I(s_1) < I(s_2) \\ 0 & \text{otherwise} \end{cases}$$

4. Concatenate all bits into a binary string.

Advantages of binary descriptors

- **Compact:** The descriptor is a short bit string (e.g., 256 bits for ORB).
- **Fast to compute:** Relies on simple intensity comparisons.
- **Fast to compare:** The distance between two binary descriptors is the **Hamming distance**:

$$d_{\text{Hamming}}(B_1, B_2) = \text{sum}(\text{xor}(B_1, B_2))$$

6.6.2 ORB: Oriented FAST and Rotated BRIEF

ORB is a binary descriptor that improves upon **BRIEF** by adding **rotation invariance** and learning optimal sampling pairs.

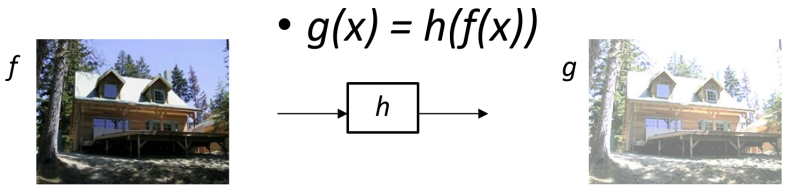
- Estimate the CoM and main orientation of the patch.
- Learn a set of 256 pairs from a training database, selecting pairs that are **uncorrelated** (to add new information) and have **high variance** (to make the feature more discriminative).
- The set of pairs is rotated by θ around the CoM before performing intensity comparisons.

7 Transformation and alignment

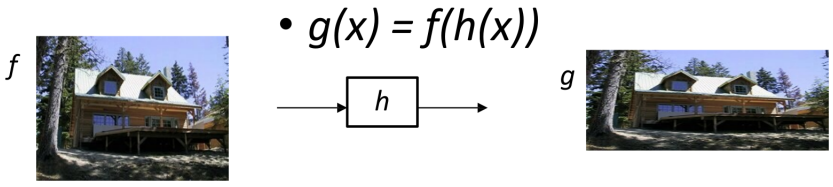
7.1 Image warping

Image processing can be categorized into two main types:

- **Image filtering:** Changes the pixel values (*range*) of an image with an operation of the form $g(x) = h(f(x))$, where f is the image and h is a function on the pixel values.



- **Image warping:** Changes the pixel coordinates (*domain*) of an image with an operation of the form $g(x) = f(h(x))$, where h is **coordinate transformation**.



We focus on *global warping*, where a single transformation $\mathbf{p}' = T(\mathbf{p})$ defined by some parameters is applied to all points $\mathbf{p} = (x, y)$ in the image.

7.2 Types of 2D transformations

7.2.1 Linear transformations

Linear transformations in 2D can be represented by a 2x2 matrix multiplication:

$$\mathbf{p}' = \mathbf{T}\mathbf{p} \quad \Longrightarrow \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Examples include:

- **Scale:** Stretches or shrinks the image. $\mathbf{S} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$
- **Rotation:** Rotates the image around the origin by an angle θ. $\mathbf{R} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$.
- **Shear:** Skews the image. $\mathbf{Sh} = \begin{bmatrix} 1 & sh_x \\ sh_y & 1 \end{bmatrix}$

Properties of 2D linear transformations

- The origin $(0, 0)$ always maps to the origin.
- Lines map to lines.
- Parallel lines remain parallel.
- Ratios of lengths along a line are preserved.
- They are closed under composition.

Limitation of 2x2 matrices

Translation, defined as $x' = x + t_x$ and $y' = y + t_y$, is **not** a linear operation on 2D coordinates and cannot be represented by a 2x2 matrix. Indeed, translation by (t_x, t_y) maps the origin $(0, 0)$ to (t_x, t_y), making the origin move.

7.2.2 Affine transformations

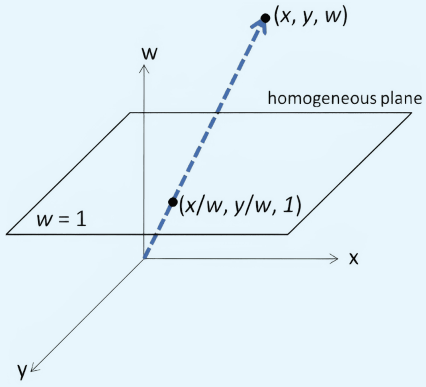
We can include translations with **homogeneous coordinates** by representing a 2D point (x, y) with a 3D vector $(x, y, 1)$.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$

The other linear transformations follow the same trick.

From homogenous coordinates to 2D

A point in homogeneous coordinates (x, y, w) corresponds to the 2D Cartesian point $(x/w, y/w)$. This process is called **normalization** of homogeneous coordinates.



General affine form A 2D *affine transformation* is any transformation represented by a 3x3 matrix where the last row is $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$. It combines linear transformations and translations.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Properties of 2D affine transformations

Origin does not necessarily map to origin (we've seen it can be translated). The other properties are analogous to 2D linear transformations.

7.2.3 Projective transformations (homographies)

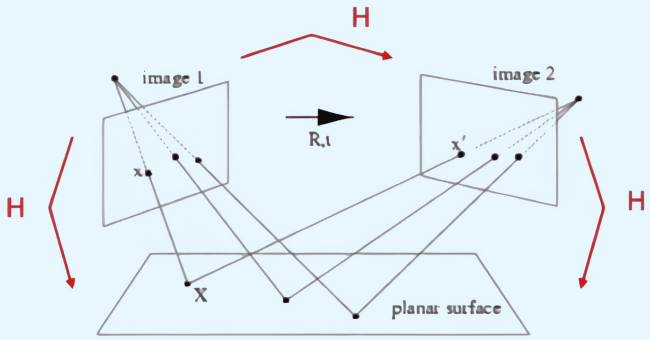
A 2D *projective transformation* (**homography**) is a linear transformation that maps points from one plane to another, and it is represented by a general 3x3 matrix **H** (defined up to a scale factor).

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \underbrace{\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}}_{\mathbf{H}} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

where the resulting Cartesian coordinates are $(x'/w', y'/w')$.

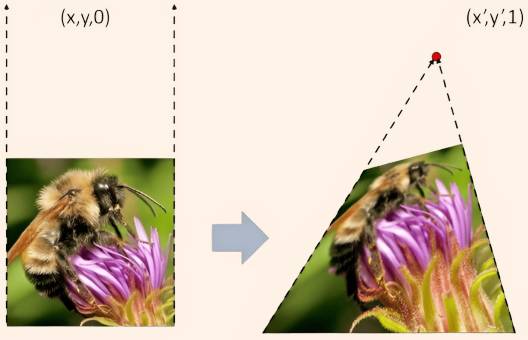
Homography application

A homography maps points between two different images of the same 3D plane.



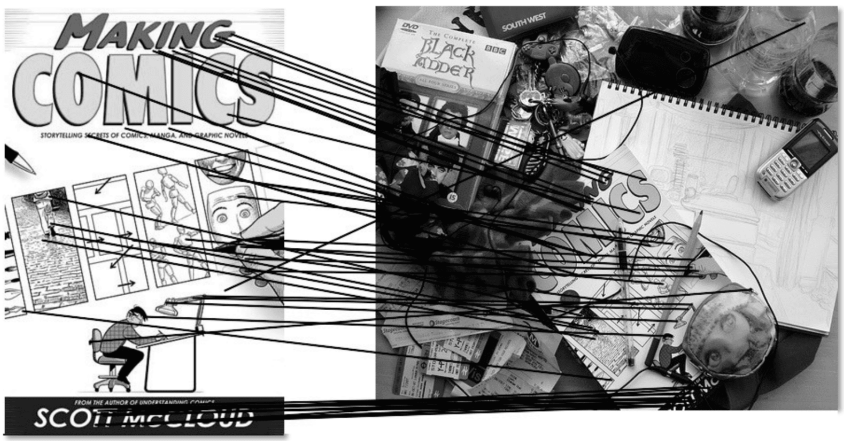
Properties of 2D projective transformations

Parallel lines do not necessarily remain parallel, as they may converge to a vanishing point. Also, **ratios are not preserved**. The other properties are the same as 2D affine transformations.



7.3 Computing transformations

Given a set of point correspondences $\{\mathbf{p}_i \leftrightarrow \mathbf{p}'_i\}$ between two images, we want to find the transformation $\mathbf{T}$ that best maps each $\mathbf{p}_i$ to its corresponding $\mathbf{p}'_i$.



7.3.1 Computing a linear transformation

For a 2D linear transformation

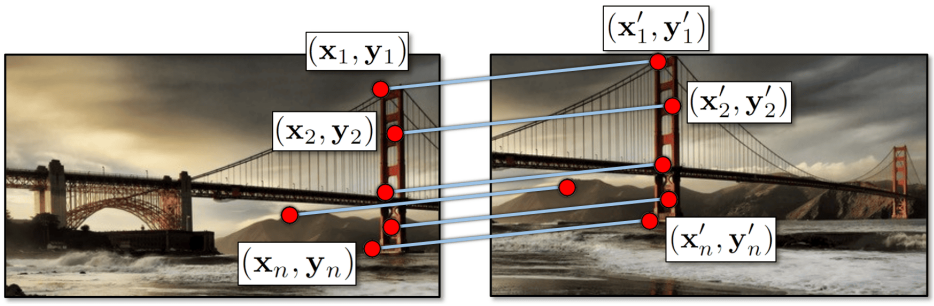
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad \Longrightarrow \quad \begin{cases} x' = ax + by \\ y' = cx + dy \end{cases}$$

where the unknowns are the element of $\mathbf{T}$, so a, b, c, d. Each point match $(x_i, y_i) \rightarrow (x_i', y_i')$ gives two independent linear equations. To uniquely solve for 4 unknowns, we need at least 4 equations, namely at least 2 point matches.

However, in practice we have more than just 2 points to match, so we end up with an **overdetermined system At = b**. To find the best fit, we employ *linear least squares*, which minimizes the sum of squared errors $||\mathbf{At} - \mathbf{b}||^2$ by solving the *normal equations*:

$$\mathbf{A}^T\mathbf{At} = \mathbf{A}^T\mathbf{b} \quad \Longrightarrow \quad \mathbf{t} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{b} = \mathbf{A}^{\#}\mathbf{b}$$

which require $\mathbf{A}^T\mathbf{A}$ to be invertible.



7.3.2 Computing an affine transformation

An affine transformation has 6 unknowns ($\{a, b, c, d, e, f\}$), and each point match provides two equations:

$$x_i' = ax_i + by_i + c$$
$$y_i' = dx_i + ey_i + f$$

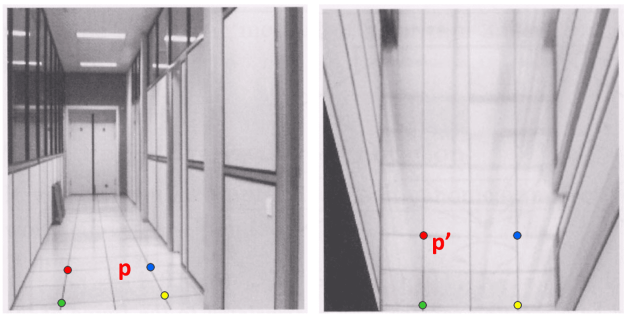
Therefore, we need **at least 3 point matches** to solve for the transformation. With $n \geq 3$ matches, we can set up a least squares problem $\mathbf{At} = \mathbf{b}$:

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n & y_n & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_n & y_n & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ \vdots \\ x'_n \\ y'_n \end{bmatrix}$$

The equation is solved for the parameter vector $\mathbf{t}$ using the normal equations.

7.3.3 Computing a homography

A homography relates points by $\mathbf{p}' \cong \mathbf{Hp}$.



For a point match $(x_i, y_i) \rightarrow (x'_i, y'_i)$, we have:

$$x'_i = \frac{h_{00}x_i + h_{01}y_i + h_{02}}{h_{20}x_i + h_{21}y_i + h_{22}} \quad , \quad y'_i = \frac{h_{10}x_i + h_{11}y_i + h_{12}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

which are not linear, but by rearranging terms, we end up with a linear system $\mathbf{Ah} = \mathbf{0}$:

$$x'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{00}x_i + h_{01}y_i + h_{02}$$
$$y'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{10}x_i + h_{11}y_i + h_{12}$$

Writing these in a linear form with unknowns $\mathbf{h}$:

$$(h_{00}x_i + h_{01}y_i + h_{02}) - x'_i(h_{20}x_i + h_{21}y_i + h_{22}) = 0$$
$$(h_{10}x_i + h_{11}y_i + h_{12}) - y'_i(h_{20}x_i + h_{21}y_i + h_{22}) = 0$$

Each point match gives two rows of $\mathbf{A}$ for the system $\mathbf{Ah} = \mathbf{0}$:

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -x'_i x_i & -x'_i y_i & -x'_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -y'_i x_i & -y'_i y_i & -y'_i \end{bmatrix} \mathbf{h} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Since $\mathbf{H}$ is defined up to a scale factor, there are 8 independent unknowns. Each point match gives two linear equations. With 4 point matches, we get 8 equations, which is the minimum sufficient to solve for $\mathbf{h}$ up to a scale factor.

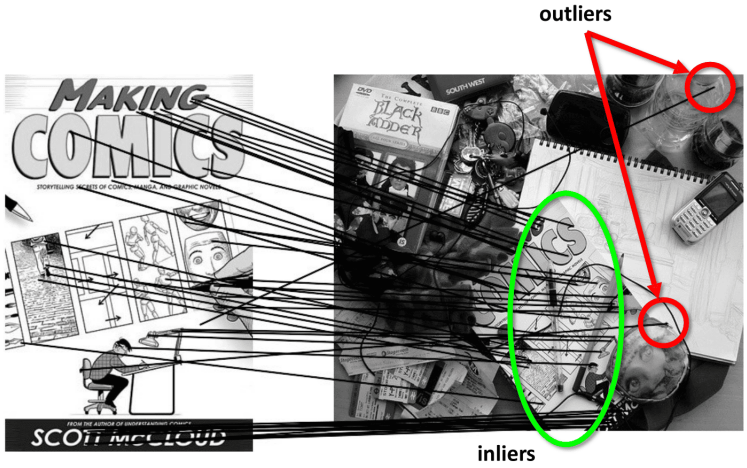
We need to solve the homogeneous system $\mathbf{Ah} = \mathbf{0}$, that is a constrained least squares problem: minimize $||\mathbf{Ah}||^2$ subject to $||\mathbf{h}|| = 1$.

Solving for homography

The solution $\mathbf{h}$ is the eigenvector of $\mathbf{A}^T\mathbf{A}$ corresponding to the smallest eigenvalue (**SVD**).

7.4 Robust estimation with RANSAC

Least squares is **sensitive to outliers** (incorrect feature matches), so we use **RANSAC** that is an iterative algorithm estimating model parameters by randomly sampling data and identifying the **largest set of inliers**.



General RANSAC Algorithm

1. **Sample** a minimal number of s points required to determine the model. (2 for linear, 3 for affine, $s = 4$ for a homography).
2. **Compute** the model parameters from this minimal sample (e.g., compute $\mathbf{H}$).
3. **Count** the number of inliers within tolerance ϵ from the prediction.
4. **Repeat** steps 1-3 for N iterations and take the model with the largest number of inliers.
5. **Re-estimate** the model parameters using LS.

RANSAC iterations

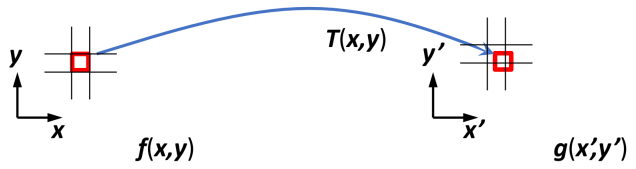
The number of iterations N needed to ensure that a good sample is chosen with probability p depends on the sample size s and the outlier ratio e:

$$N \geq \frac{\log(1-p)}{\log(1-(1-e)^s)}$$

7.5 Image warping implementation and blending

7.5.1 Forward vs. inverse warping

- **Forward warping:** Iterate over each pixel (x, y) in the source image, compute its new position $(x', y') = T(x, y)$, and copy the pixel value. **Problems:** can result in holes (unfilled pixels) and overlaps (multiple source pixels mapping to one destination pixel) in the output image.



- **Inverse warping:** Iterate over each pixel (x', y') in the destination image, compute its corresponding position in the source image $(x, y) = T^{-1}(x', y')$, and sample the color. This ensures every pixel in the output is filled.

Since the calculated source position (x, y) is often not at an integer coordinate, **interpolation** is required (e.g., nearest neighbor, bilinear, bicubic).

7.5.2 Image blending

After images are aligned, they need to be blended seamlessly.

- **Feathering:** A simple approach that uses a weighted average (alpha blending) in the overlapping regions. The weights transition smoothly from one image to the other. Can produce "ghosting" artifacts if alignment is imperfect.
- **Pyramid blending:** Seen in Section 2.3.1.

8 Recognition

8.1 Recognition tasks

Visual recognition encompasses several fundamental tasks with increasing complexity: **image classification**, **object detection**, **semantic segmentation**, and **instance segmentation**.

8.2 Image classification

We use machine learning approaches with large-scale labeled datasets like ImageNet. *Transfer learning* allows for **pre-training** a model on a large dataset, and then **fine-tuning** it on smaller task-specific dataset.

8.2.1 Bag-of-Words (BoW) model

BoW achieves **spatial invariance** by representing images as histograms of local features, ignoring their spatial arrangement.

BoW pipeline

1. Detect and describe features (e.g., using SIFT).
2. Cluster all feature vectors from a training set (e.g., using k-means to form the visual vocabulary.
3. Assign each feature descriptor to its nearest visual word in the vocabulary.
4. Represent each image by the frequency histogram of its assigned visual words.
5. Train a standard classifier (e.g., SVM) on the histogram representations.

8.3 Convolutional Neural Networks (CNNs)

CNNs consist of 3 main types of layers.

- **Convolutional layers:** Apply a set of learnable filters across the input feature map. They leverage **weight sharing** to reduce the number of learnable parameters of FC layers.
- **Downsampling (pooling) Layers:** Reduce the spatial resolution of the feature maps to increase the receptive field of subsequent neurons.
- **FC layers:** Used at the end of the network to perform the final classification based on the features extracted by the convolutional layers.

8.3.1 Output and loss function

For multi-class classification, the final layer is a **softmax** function, which converts logits into a valid probability distribution over the classes. The network is trained by minimizing the **cross-entropy (CE) loss**, which measures the dissimilarity between the predictions and the true labels.

8.3.2 Resnet

Resnet enabled the training of deep networks by introducing **residual connections**, allowing for the gradient to bypass layers, mitigating the vanishing gradient problem.

8.4 Semantic segmentation

Semantic segmentation performs pixel-level predictions using the **encoder-decoder architecture**.

- The *encoder* is a classification network (e.g., ResNet) that progressively downsamples the input to learn high-level features.
- The *decoder* progressively upsamples these features to produce a segmentation map.
- **Skip connections** are crucial for combining coarse information from the decoder with fine information from the encoder (see **U-Net**).

8.5 Object detection

The main evaluation metirc is *mean average precision* (**MAP**) calculated from precision-recall curves where detections are deemed correct if their interesection-over-union with a ground-truth box is ≥ 0.5.

8.5.1 Two-stage detectors

These methods first propose candidate regions and then classify them.

- **R-CNN (2014):** Used Selective Search to generate 2k region proposals and ran a CNN on each one independently. Slow.
- **Fast R-CNN (2015):** Ran the CNN once on the entire image to create a feature map. Region proposals were then projected onto this map, and a special **RoI Pooling** layer extracted fixed-size features for each proposal. Faster.
- **Faster R-CNN (2015):** Replaced the slow external region proposal method with a learnable **Region Proposal Network (RPN)**. The RPN slides over the feature map and predicts objectness and box refinements relative to a set of pre-defined **anchor boxes** of various scales and aspect ratios. Much faster.

8.5.2 Single-stage detectors

Frameworks like **YOLO** and **SSD** discard the region proposal stage and predict bounding boxes and class probabilities directly from the feature map in a single pass. They are generally faster but can be less accurate than two-stage methods.

8.5.3 Feature Pyramid Networks (FPN)

FPNs recognize objects at multiple scale by constructing a **multi-scale feature pyramid** within the network. A top-down path with lateral connections enriches low-resolution features with high-resolution features, improving detection performance.

8.6 Extending the recognition framework

The Faster R-CNN framework is highly flexible and can be extended to solve other tasks by adding new prediction "heads."

8.6.1 Instance segmentation with Mask R-CNN

Mask R-CNN extends Faster R-CNN by adding a parallel branch that predicts a pixel-level segmentation mask for each detected object. It replaces RoI Pooling with a more precise **RoIAlign** layer to preserve spatial alignment for accurate mask prediction.

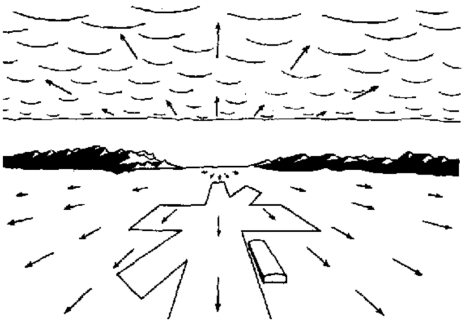
8.6.2 Panoptic segmentation

This task unifies semantic segmentation (for "stuff" like sky, road) and instance segmentation (for "things" like cars, people) to produce a single, comprehensive scene understanding output.

9 Optical flow

9.1 Introduction to optical flow

Optical flow is the **apparent motion** of objects, surfaces, and edges in a visual scene caused by the relative motion between an observer (the camera) and a scene. It is a **2D vector field** where each vector is a **pixel displacement** between two subsequent frames.

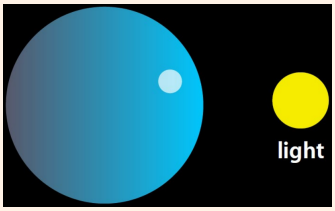


Motion field vs. optical flow

The **motion field** is the true 2D projection of the 3D motion of scene points onto the image plane. The **optical flow** is the apparent motion of brightness patterns in the image, so it is an approximation of the motion field, but they can differ.

- A rotating, uniformly lit Lambertian sphere has a **motion field**, but zero optical flow since the appearance does not change.

- A stationary specular sphere with a moving light source has zero motion field, but the reflection's movement creates **optical flow**.

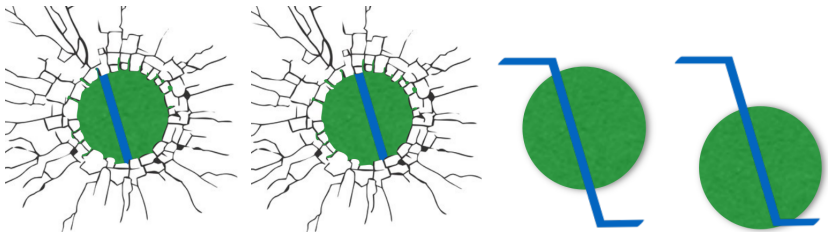


Stereo vs. optical flow

- **Stereo:**
  - Uses two images from **different viewpoints** (two cameras) taken **at the same time**, used for 3D depth estimation.
  - The correspondence problem is **1D** due to epipolar constraints: for a point in one image, its corresponding point in the other image is constrained to lie along a specific epipolar line, significantly reducing the search space.
- **Optical Flow:**
  - Uses two images from the **same viewpoint** (a single camera) taken at **different times**, used to infer object velocity or camera motion.
  - It's a **2D estimation problem** where each pixel's apparent displacement vector can be in any direction and magnitude. This means objects can move freely in 3D space, and their projection onto the 2D image plane results in potentially unconstrained 2D pixel shifts. This means that for each pixel in the first frame, to find its corresponding location in the second frame, we need to search in all directions (horizontal, vertical, and diagonals) and across various distances within a given search window.

9.1.1 The aperture problem

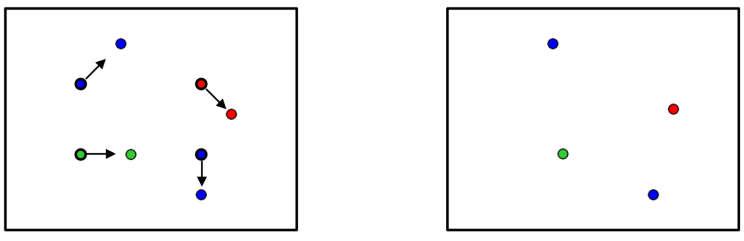
When viewing a moving object through a small aperture, equivalent to looking at a **local image patch**, its motion can be ambiguous. For a moving edge, only the motion perpendicular to the edge can be determined.



9.2 Estimating optical flow

To estimate the optical flow field $(u(x,y), v(x,y))$ between two subsequent frames, we make several assumptions.

- **Brightness constancy:** the brightness $(I(x(t), y(t), t)$ of a point remains constant over time.
- **Small motion:** points do not move too far.
- **Spatial coherence:** points move like their neighbors.



9.2.1 The brightness constancy equation

Based on our assumptions, can relate the pixel intensity at position (x, y) at time t to its new position at time $t + \delta t$ by means of the *brightness constancy equation*:

$$I(x, y, t) = I(x + u\delta t, y + v\delta t, t + \delta t)$$

where (u, v) is the **optical flow velocity vector** at pixel location (x, y) at time t, describing how fast and in what direction that pixel is moving: u is the instantaneous velocity of the pixel in the x-direction (horizontal), while v is its instantaneous velocity in the y-direction (vertical). For a small δt, we can linearize the right-hand side using a first-order Taylor series expansion:

$$I(x + u\delta t, y + v\delta t, t + \delta t) \approx I(x, y, t) + \frac{\partial I}{\partial x}u\delta t + \frac{\partial I}{\partial y}v\delta t + \frac{\partial I}{\partial t}\delta t$$

which, substituted back gives the *linearized brightness constancy equation*:

$$I(x, y, t) = I(x, y, t) + \frac{\partial I}{\partial x}u\delta t + \frac{\partial I}{\partial y}v\delta t + \frac{\partial I}{\partial t}\delta t$$

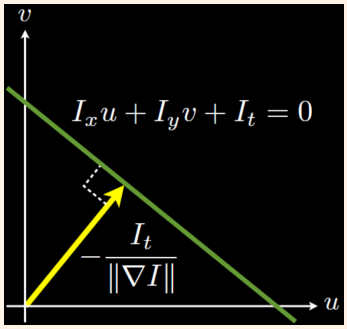
We can cancel $I(x, y, t)$ from both sides and divide by δt to get the *optical flow constraint equation*:

$$\underbrace{I_x u}_{x\text{-flow}} + \underbrace{I_y v}_{y\text{-flow}} + I_t = 0$$

where I_x, I_y are the spatial **image gradients** (Sobel, DOG, ...) and I_t is the **temporal gradient** (frame-to-frame difference).

Underconstrained problem

The optical flow constraint gives us only **one equation per pixel** but we need to solve for **two unknowns** (u, v). This is an underconstrained problem: there are ∞^1 solutions ((u, v) pairs) satisfying this single equation $\implies$ the solutions lie on a line in the (u, v) space. This is why the aperture problem exists for individual pixels or small image patches: we can't uniquely determine the full 2D motion with just local gradient information.



9.3 Lucas-Kanade optical flow

The Lucas-Kanade method introduces an additional constraint by using the **spatial coherence** assumption: it assumes that the flow (u, v) is constant within a small patch around a pixel, so as to pool information from multiple pixels.

9.3.1 Lucas-Kanade solution

Instead of having one equation with two unknowns per pixel, we now have N^2 equations for an $N \times N$ patch:

$$\begin{aligned} I_x(\mathbf{p}_1)u + I_y(\mathbf{p}_1)v &= -I_t(\mathbf{p}_1) \\ I_x(\mathbf{p}_2)u + I_y(\mathbf{p}_2)v &= -I_t(\mathbf{p}_2) \\ &\vdots \\ I_x(\mathbf{p}_{N^2})u + I_y(\mathbf{p}_{N^2})v &= -I_t(\mathbf{p}_{N^2}) \end{aligned}$$

This is an **overdetermined system** of the form $\mathbf{Ax} = \mathbf{b}$, which can be solved using linear least squares:

$$\underbrace{\begin{bmatrix} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ \vdots & \vdots \\ I_x(\mathbf{p}_{N^2}) & I_y(\mathbf{p}_{N^2}) \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} u \\ v \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} -I_t(\mathbf{p}_1) \\ \vdots \\ -I_t(\mathbf{p}_{N^2}) \end{bmatrix}}_{\mathbf{b}}$$

where $\mathbf{A} \in \mathbb{R}^{N^2 \times 2}, \mathbf{x} \in \mathbb{R}^2, \mathbf{b} \in \mathbb{R}^{N^2}$. The least-squares solution is $\mathbf{x} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{b} = \mathbf{A}^{\#}\mathbf{b}$. The matrix $\mathbf{A}^T\mathbf{A}$ is the **second-moment matrix**, familiar from the Harris corner detector:

$$\mathbf{A}^T\mathbf{A} = \sum_{\mathbf{p} \in \text{patch}} \begin{bmatrix} I_x^2 & I_xI_y \\ I_xI_y & I_y^2 \end{bmatrix}$$

When does Lucas-Kanade work?

For the solution to exist and be reliable, the matrix $\mathbf{A}^T\mathbf{A}$ must be invertible and well-conditioned. This depends on its eigenvalues, λ_1 and λ_2.

- **Flat region:** $I_x, I_y \approx 0 \implies \lambda_1, \lambda_2$ are small. Matrix is ill-conditioned.
- **Edge:** Gradient is strong in one direction $\implies \lambda_1 \gg \lambda_2 \approx 0$. Matrix is ill-conditioned (aperture problem).
- **Corner/Texture:** Gradients are strong in multiple directions $\implies \lambda_1, \lambda_2$ are both large. Matrix is well-conditioned and flow can be reliably computed.

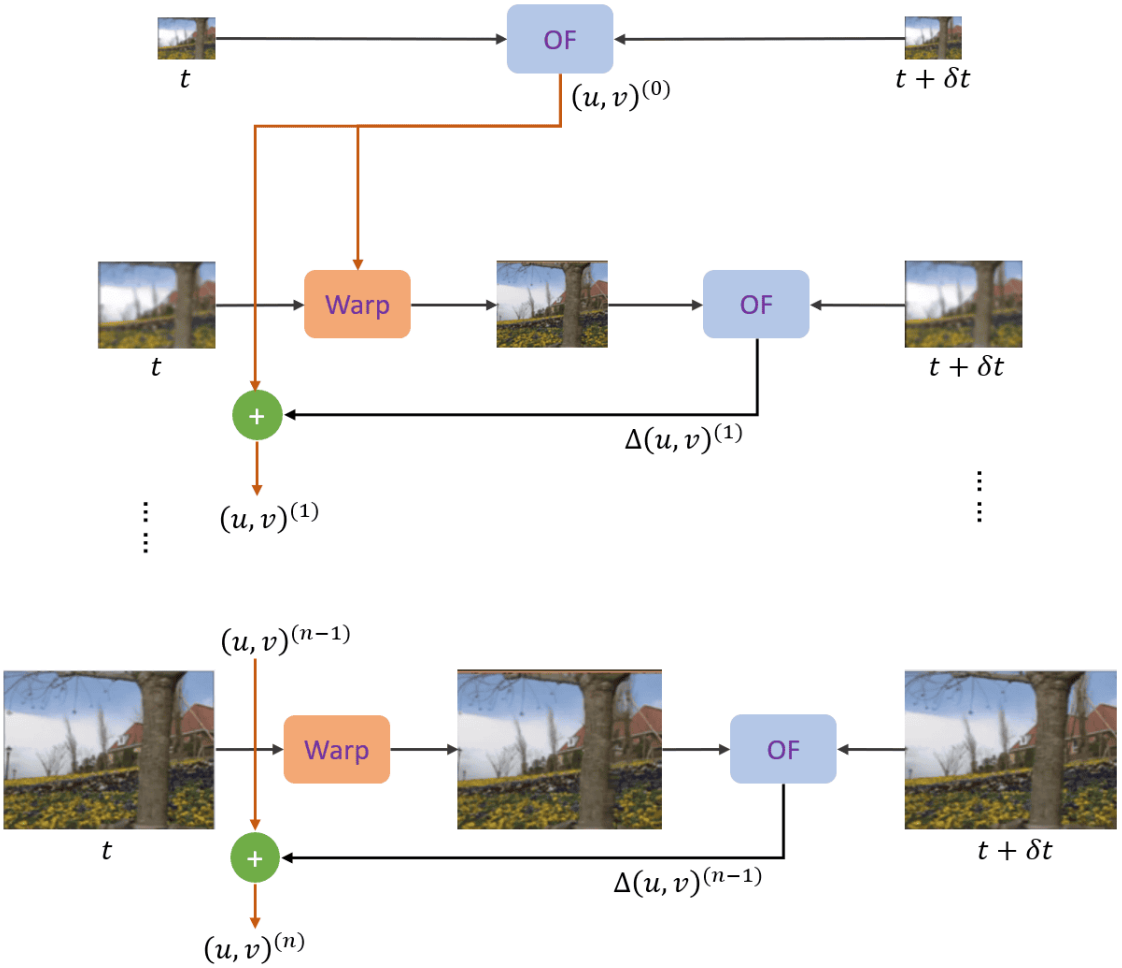
9.3.2 Handling large motions

The small motion assumption is violated when displacements are large. This can be addressed with a **coarse-to-fine estimation strategy** using image pyramids.

1. **Pyramid construction:** Build Gaussian pyramids for both frames (I_t and I_{t+1}).
2. **Coarsest level estimation:** Estimate $(u, v)^{(0)}$ with LK at G_N because large motions appear as small displacements
3. **Iterative refinement:**
   - **Upsample flow:** Upsample $(u, v)^{(0)}$ to project it to G_{N-1}.
   - **Warp frame:** Use it to warp I_{t+1} towards I_t (**inverse warping**), removing most motion. rresponds to pixel $(x + u\delta t, y + v\delta t)$ in image I_{t+1}.
   - **Compute residual:** Compute **residual optical flow** $\Delta(u, v)^{(1)}$ between I_t and I_{t+1}.

- **Accumulate flow:** Add it to the upsampled flow to get the total flow for the current level $(u, v)^{(1)} = (u, v)^{(0)} + \Delta(u, v)^{(1)}$.

4. **Full resolution:** Repeat step 3 until G_0 to get $(u, v)^{(N)}$.



9.4 Horn-Schunck optical flow

Horn-Schunck assumes the optical flow is **smooth** across the entire image. It estimates it by iteratively minimizing:

$$E(u, v) = \underbrace{\iint (I_x u + I_y v + I_t)^2 dx dy}_{\text{Data term (Brightness Constancy)}} + \lambda \underbrace{\iint (\|\nabla u\|^2 + \|\nabla v\|^2) dx dy}_{\text{Smoothness term}}$$

The data term enforces the **brightness constancy constraint**, while the smoothness term **penalizes large variations** in the flow field, controlled by the regularization parameter λ.

Oversmoothing

The quadratic penalty used by Horn-Schnuck assumes Gaussian noise models, which are not robust to outliers that would penalize heavily large changes in flow, causing **oversmoothing** at object boundaries.

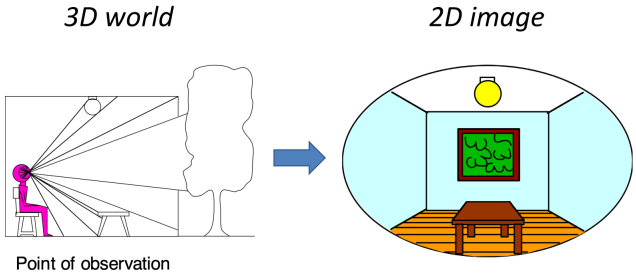
9.4.1 Robust regularization

We replace quadratic penalties with **robust loss functions** that are less sensitive to large errors, allowing for sharp changes at object boundaries while still smoothing other regions.

10 Camera calibration

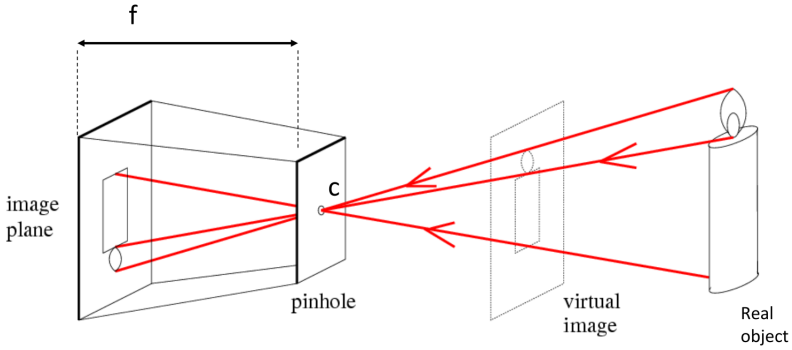
10.1 The camera model

A *camera* acts as a **dimensionality reduction machine**, performing a **projective transformation** to map the 3D world onto a 2D image plane. *Camera calibration* is the process of computing the parameters of this transformation.



10.1.1 The pinhole camera

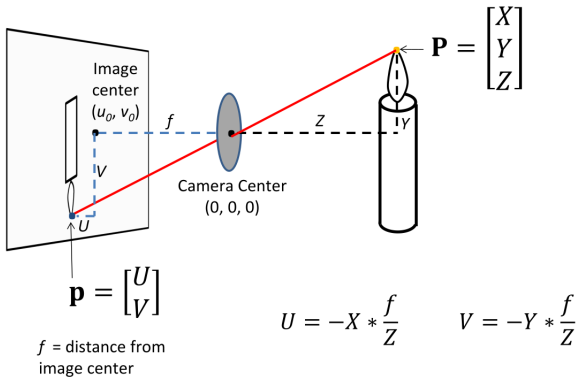
The simplest camera model is the **pinhole camera**, which uses a tiny aperture to block most light rays, allowing only a single ray from each scene point to reach the image sensor. This prevents blurring and forms an inverted image.



Perspective projection Using similar triangles, we can derive the relationship between a 3D point $\mathbf{x}_c = (x_c, y_c, z_c)$ in the camera's coordinate system and its 2D projection (x_i, y_i) on the image plane:

$$\frac{x_i}{f} = \frac{x_c}{z_c} \implies x_i = f\frac{x_c}{z_c} \quad \text{and} \quad \frac{y_i}{f} = \frac{y_c}{z_c} \implies y_i = f\frac{y_c}{z_c}$$

where f is the focal length. This is the **normalization step** to project 3D point onto a plane at $z = 1$.



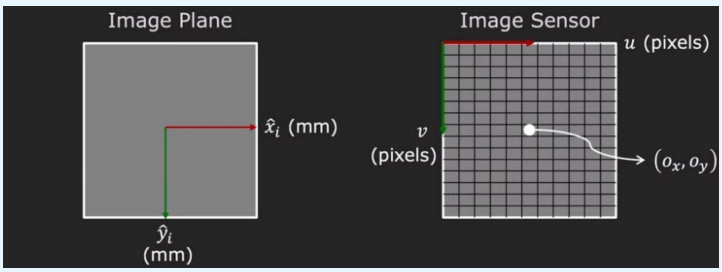
10.1.2 From image plane to pixels

The physical image plane (in mm) is mapped to a digital sensor grid (in pixels). This involves:

- **Scaling**: A scaling factor for **pixel density** (m_x, m_y pixels/mm): m_x is the number of pixels per millimeter in the horizontal direction on the sensor, whereas m_y in the vertial direction. They are the **unknowns** of the calibration process.
- **Translation**: An offset for the **principal point** (o_x, o_y), which is where the optical axis intersects the sensor/image plane. This represents the pixel offset from the principal point to the top-left corner of the digital image sensor grid, which is the origin for pixel coordinates.

Digital sensor positioning

The digital sensor is physically positioned at the **image plane**. Conceptually, for the equations $x_i = f(x_c/z_c)$, the image plane is placed f units in front of the camera center along the z-axis $\implies$ the digital sensor is f units in front of the camera center along the optical axis.



The final pixel coordinates (u, v) are given by:

$$u = m_x \underbrace{\left(f\frac{x_c}{z_c}\right)}_{x_i} + o_x \quad \text{and} \quad v = m_y \underbrace{\left(f\frac{y_c}{z_c}\right)}_{y_i} + o_y$$

We can group the parameters into focal lengths in pixels, $f_x = m_x f$ and $f_y = m_y f$:

$$u = f_x\frac{x_c}{z_c} + o_x \quad \text{and} \quad v = f_y\frac{y_c}{z_c} + o_y$$

10.1.3 The camera matrix

To create a complete linear model, we combine the camera's parameters into a single **projection matrix P**. This requires using homogeneous coordinates for both 3D world points and 2D image points. The full transformation from a 3D world point $\tilde{\mathbf{x}}_w$ to a 2D pixel $\tilde{\mathbf{u}}$ is:

$$\tilde{\mathbf{u}} = \mathbf{P}\tilde{\mathbf{x}}_w = \mathbf{K}[\mathbf{R}|\mathbf{t}]\tilde{\mathbf{x}}_w$$

This model is composed of two main parts:

Intrinsic matrix K Describes the camera's internal geometry, mapping 3D camera coordinates to 2D pixel coordinates.

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

The parameters $\{f_x, f_y, o_x, o_y\}$ are the **intrinsic parameters**.

Extrinsic matrix $[\mathbf{R}|\mathbf{t}]$ Describes the camera's pose in the world, mapping 3D world coordinates to 3D camera coordinates.

- $\mathbf{R}$: a 3×3 orthonormal rotation matrix.
- $\mathbf{t}$: a 3×1 translation vector.

$$[\mathbf{R}|\mathbf{t}] = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}$$

These are the **extrinsic parameters**.

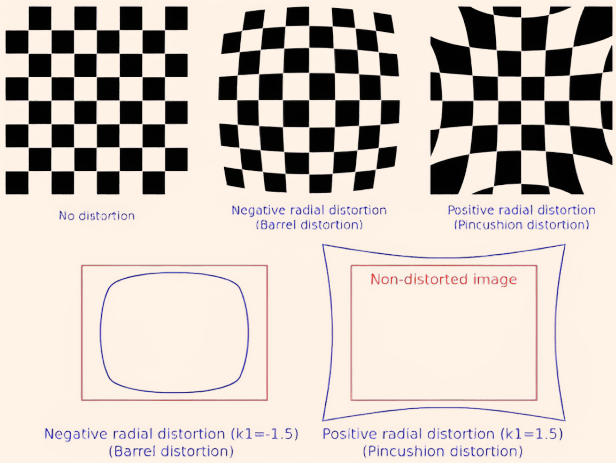
Full transformation In **homogeneous coordinates**, the full transformation from 3D world coordinates to 2D pixel coordinates is:

$$\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \mathbf{M}_{\text{int}} \left[\begin{array}{c|c} \mathbf{R} & \mathbf{t} \\ \hline \mathbf{0}^T & 1 \end{array}\right] \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix} = \mathbf{M}_{\text{int}}\mathbf{M}_{\text{ext}}\tilde{\mathbf{x}}_w = \mathbf{P}\tilde{\mathbf{x}}_w$$

We can use $[\mathbf{R} \mid \mathbf{t}] \in \mathbb{R}^{3\times4}$ (rather than $\mathbf{M}_{\text{ext}} \in \mathbb{R}^{4\times4}$) and obtain $\mathbf{x}_c \in \mathbb{R}^3$, that can be then scaled to normalized camera coordinates as $(x_c/z_c, y_c/z_c, 1)$ and used to compute $\tilde{\mathbf{u}}$ with $\mathbf{K} \in \mathbb{R}^{3\times3}$ (rather than $\mathbf{M}_{\text{int}} \in \mathbb{R}^{3\times4}$), that again can be scaled to obtain the final 2D pixel coordinates $(\tilde{u}/\tilde{w}, \tilde{v}/\tilde{w}$.

Lens distortion

Real lenses introduce radial and tangential distortions.



We can estimate the distortion coefficient as intrinsic parameters to "undistort" the image by setting $x = x_c/z_c, y = y_c/z_c, r^2 = x^2 + y^2$:

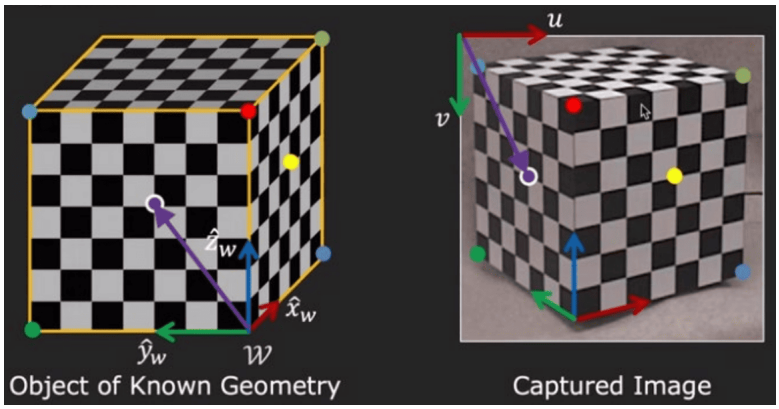
$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{(1 + \kappa_1 r^2 + \kappa_2 r^4)}_{\text{radial distortion}} \begin{pmatrix} x \\ y \end{pmatrix} + \underbrace{\begin{pmatrix} 2\kappa_3 xy + \kappa_4(r^2 + 2x^2) \\ 2\kappa_4 xy + \kappa_3(r^2 + 2y^2) \end{pmatrix}}_{\text{tangential distortion}}$$

Then scale to get pixel coordinates:

$$\begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} f_x x' + c_x \\ f_y y' + c_y \end{pmatrix}$$

10.2 Camera calibration

Calibration is the process of estimating the 3×4 projection matrix $\mathbf{P}$. This can be done by observing an object of known geometry, and identifying at least 6 pairs (cause $\mathbf{P}$ is defined up to a scale factor, so there are 11 unknowns) of known 3D world points $\mathbf{x}_w^{(i)}$ and their corresponding 2D image points $\mathbf{u}^{(i)}$.



This is because each point correspondence $\mathbf{x}_w^{(i)} \leftrightarrow \mathbf{u}^{(i)}$ provides:

$$\begin{bmatrix} u^{(i)} \\ v^{(i)} \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w^{(i)} \\ y_w^{(i)} \\ z_w^{(i)} \\ 1 \end{bmatrix}$$

which can be expanded as 2 linear equations:

$$u^{(i)} = \frac{p_{11}x_w^{(i)} + p_{12}y_w^{(i)} + p_{13}z_w^{(i)} + p_{14}}{p_{31}x_w^{(i)} + p_{32}y_w^{(i)} + p_{33}z_w^{(i)} + p_{34}}$$
$$v^{(i)} = \frac{p_{21}x_w^{(i)} + p_{22}y_w^{(i)} + p_{23}z_w^{(i)} + p_{24}}{p_{31}x_w^{(i)} + p_{32}y_w^{(i)} + p_{33}z_w^{(i)} + p_{34}}$$

and rearranged to form the homogeneous linear system $\mathbf{Ap} = \mathbf{0}$, which is a constrained least squares problem solved by minimizing $||\mathbf{Ap}||^2$ subject to $||\mathbf{p}|| = 1$. The solution $\mathbf{p}$ is the eigenvector of $\mathbf{A}^T\mathbf{A}$ corresponding to the smallest eigenvalue. The vector $\mathbf{p} \in \mathbb{R}^{12}$ is then reshaped into $\mathbf{P} \in \mathbb{R}^{3\times4}$.

10.2.1 Extracting intrinsic and extrinsic parameters

Once $\mathbf{P}$ is estimated, we can decompose it to find the physically meaningful parameters $\mathbf{K}$, $\mathbf{R}$, and $\mathbf{t}$.

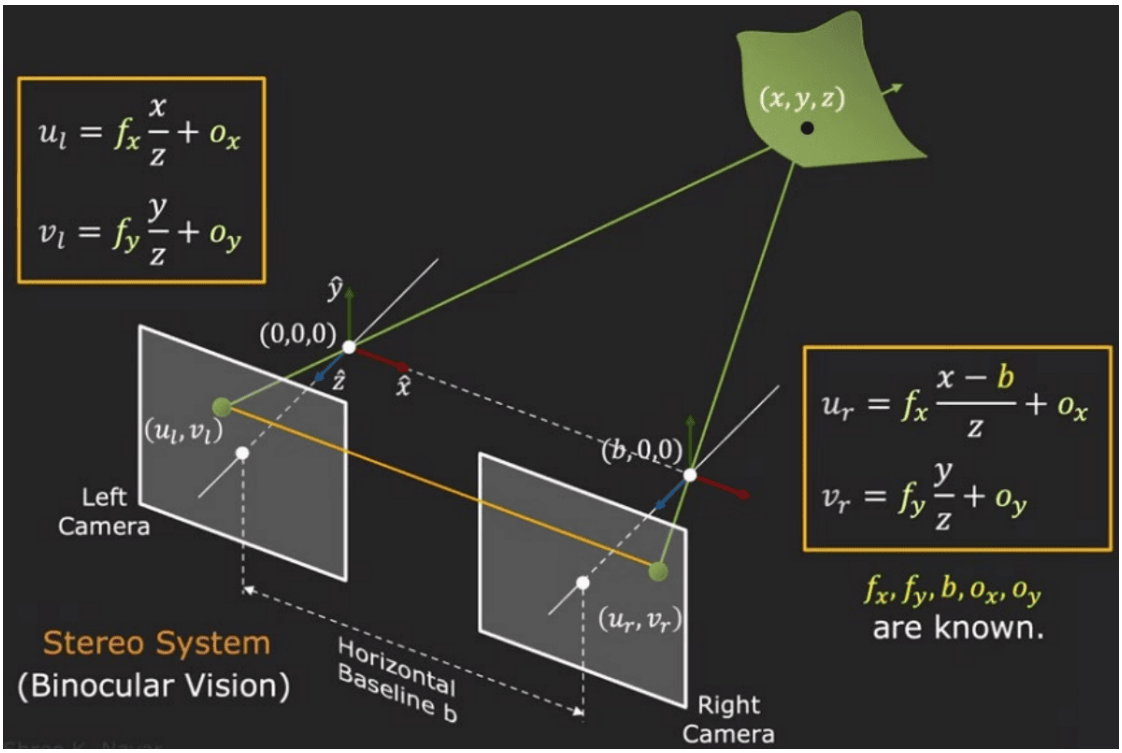
$$\begin{cases} \mathbf{P} = [\mathbf{M} \mid \mathbf{p}_4] \\ \mathbf{P} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}] = [\mathbf{KR} \mid \mathbf{Kt}] \end{cases} \implies \begin{matrix} \mathbf{M} = \mathbf{KR} \\ \mathbf{p}_4 = \mathbf{Kt} \end{matrix} \implies \begin{matrix} \mathbf{R} = \mathbf{K}^{-1}\mathbf{M} \\ \mathbf{t} = \mathbf{K}^{-1}\mathbf{p}_4 \end{matrix}$$

where $\mathbf{M}$ is the 3×3 left submatrix of $\mathbf{P}$ and $\mathbf{p}_4$ is its fourth column. We used **RQ factorization** to decouple $\mathbf{K}$ and $\mathbf{R}$ (upper-triangular $\times$ orthonormal).

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} = KR$$

10.3 Stereo vision

Stereo vision uses **two or more cameras** to infer **depth** from images by *triangulation*.



10.3.1 Depth from disparity

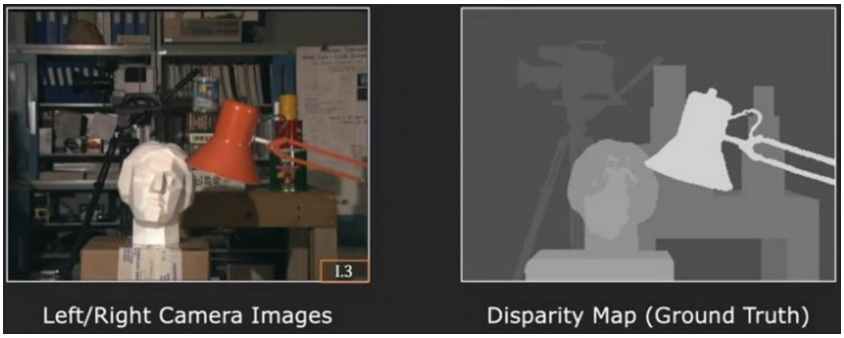
For a simple stereo rig with two parallel cameras (so $\mathbf{R} = \mathbf{I}$) separated by a **horizontal baseline** b (*rectified stereo*), the depth z of a point is inversely proportional to its *disparity* d and proportional to b.

$$z = \frac{bf_x}{d} = \frac{bf_x}{u_l - u_r}$$

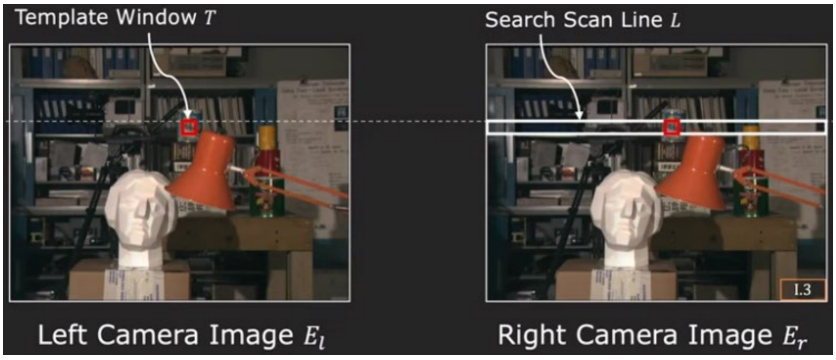
where d is the difference in the horizontal pixel coordinates of the same 3D point between the two images. A high d means the 3D point is close to the camera (large horizontal shift), while low d means it's far away (small horizontal shift).

10.3.2 Stereo matching

The *disparity map* is the **horizontal displacement** in each pixel between two images, thus we can compute it by finding the corresponding points between the two images with *stereo matching*.



For **rectified stereo pairs**, stereo matching is simplified to a 1D scan along the same horizontal image row (*scanline*), because the epipolar lines of the two images are horizontal and aligned, so a 3D point will have the same y coordinate when projected onto the two images ($v_l = v_r$).

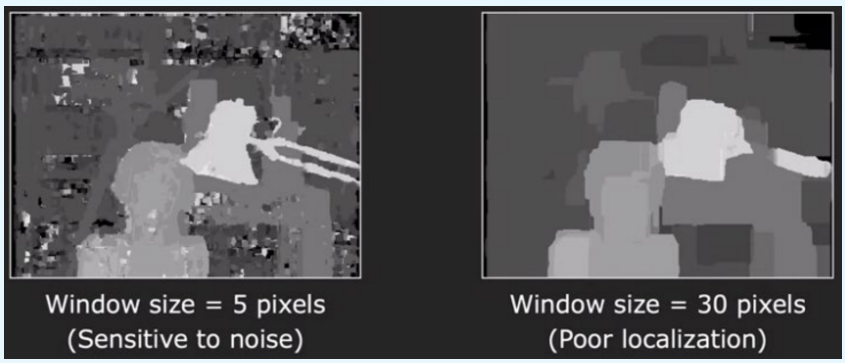


Window-based matching Local method that performs the 1D search. For each pixel in I_ℓ:

1. Take a small patch around it.
2. Compare it to patches at different positions along the same horizontal scanline in I_r.
3. Determine the best match using SSD, SAD, NCC to find the corresponding point, then compute $d = u_l - u_r$.

Window size trade-off

- **Small window**: captures fine details but produces noisy disparity map.
- **Large window**: robust to noise but blurs discontinituies $\implies$ fails at localizing thin objects.



10.3.3 Stereo as energy minimization

To overcome the limitations of local methods, stereo matching can be formulated as a global **energy minimization problem**. We seek the disparity map d that minimizes a total energy function:

$$E(d) = \underbrace{\sum_{(x,y)} E_d(x, y, d(x, y))}_{\text{Data term}} + \lambda \underbrace{\sum_{(p,q)\in\mathcal{E}} E_s(d_p, d_q)}_{\text{Smoothness term}}$$

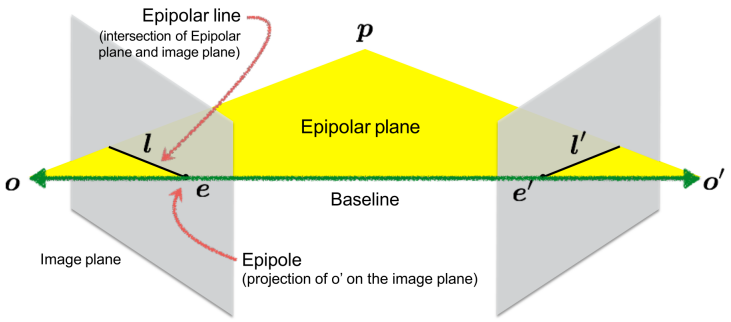
- The **data term** E_d penalizes poor matches. It is the cost (e.g., SSD) for assigning disparity $d(x, y)$ to pixel (x, y).
- The **smoothness term** E_s enforces the assumption that neighboring pixels should have similar disparities. It penalizes differences in disparity between adjacent pixels p and q.

11 Epipolar geometry

11.1 Introduction

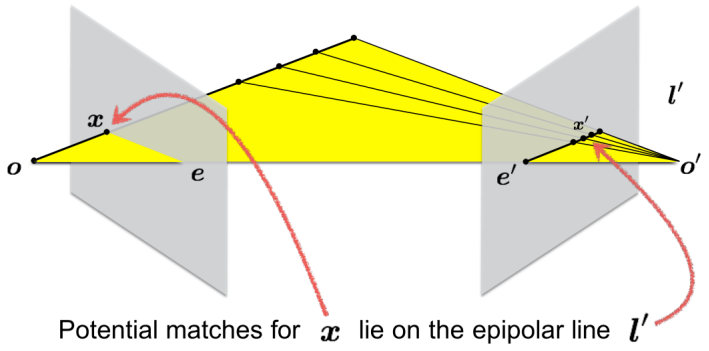
Epipolar geometry relates the 2D projections $(\mathbf{x}, \mathbf{x}')$ of a 3D point $\mathbf{X}$ observed from two different camera views, providing a geometric constraint for finding corresponding points and relative camera poses.

- **Camera centers $(\mathbf{O}, \mathbf{O}')$:** The centers of projection for the two cameras.
- **Baseline:** The line segment connecting the two camera centers $\mathbf{O}$ and $\mathbf{O}'$.
- **Epipolar plane:** The plane defined by a 3D point $\mathbf{X}$ and the two camera centers $\mathbf{O}$ and $\mathbf{O}'$.
- **Epipoles $(\mathbf{e}, \mathbf{e}')$:** The points where the baseline intersects the two image planes. The epipole $\mathbf{e}$ in the first image is the projection of the second camera's center $\mathbf{O}'$, and vice versa.
- **Epipolar Line $(\mathbf{l}, \mathbf{l}')$:** The intersection of the epipolar plane with an image plane. All epipolar lines in an image pass through the epipole. The epipolar line $\mathbf{l}$ connects $\mathbf{e}$ to the projection $\mathbf{x}$ of $\mathbf{X}$ in the first image.



11.1.1 The epipolar constraint

For a given projection $\mathbf{x}$ in the first image, its corresponding projection $\mathbf{x}'$ in the second image **must lie on $\mathbf{l}'$** because the 3D point $\mathbf{X}$ can be anywhere on the ray back-projected from $\mathbf{x}$, and the projection of this entire ray onto the second image is $\mathbf{l}'$.



The power of the epipolar constraint

This constraint reduces the search space for correspondences from the entire 2D image to a 1D line, simplifying stereo matching (especially unrectified).

11.2 The essential matrix

The *essential matrix* $\mathbf{E} \in \mathbb{R}^{3\times3}$ encodes the epipolar constraint for **calibrated** cameras ($\mathbf{K}$ known). It relates a point in one image to its corresponding epipolar line in the other ($\mathbf{Ex} = \mathbf{l}'$), leading to the fundamental epipolar constraint:

$$\boxed{\mathbf{x}'^T\mathbf{E}\mathbf{x} = 0}$$

11.2.1 Derivation

Let $\mathbf{x}, \mathbf{x}'$ be the *normalized image coordinates* of a 3D point $\mathbf{X}$ in the two camera frames, meaning they can be treated as 3D vectors from the camera's optical center.

$$\mathbf{x} = \mathbf{K}^{-1}\mathbf{p}$$

Effect of $\mathbf{K}^{-1}$

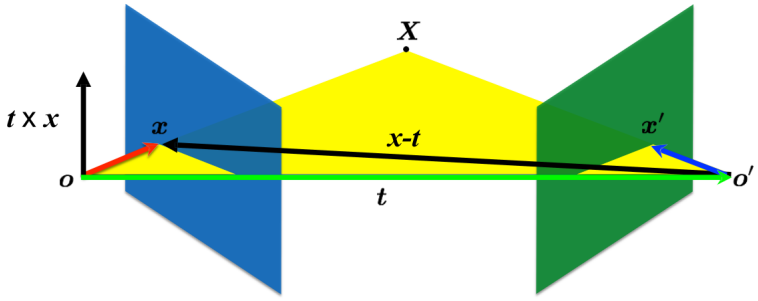
When multiplying by $\mathbf{K}^{-1}$, we are converting pixel coordinates (u, v) back to what they would be if the camera were ideal, namely a pinhole camera with $f = 1$ and a principal point at the origin, that is operating on a normalized image plane at $z = 1$.

Let $\mathbf{R}$ and $\mathbf{t}$ be the rotation and translation that transform a point from the first camera frame to the second. Here, $\mathbf{t}$ represents the position of $\mathbf{o}'$ in the first camera frame. The transformation is:

$$\mathbf{x}' = \mathbf{R}(\mathbf{x} - \mathbf{t})$$

The optical centers of the two cameras $(\mathbf{o}, \mathbf{o}')$ and the 3D point $\mathbf{X}$ are coplanar $\implies$ the following three vectors, when expressed in the **first camera frame**, are coplanar:

- The vector from $\mathbf{o}$ to $\mathbf{X}$, that is $\mathbf{x}$
- The vector from $\mathbf{o}$ to $\mathbf{o}'$, that is $\mathbf{t}$
- The vector from $\mathbf{o}'$ to $\mathbf{X}$, that is $\mathbf{x} - \mathbf{t}$



$$(\boldsymbol{x} - \boldsymbol{t})^\top (\boldsymbol{t} \times \boldsymbol{x}) = 0$$

dot product of orthogonal vectors — cross-product: vector orthogonal to plane

This geometric constraint can be expressed in the first camera frame, so considering $\mathbf{x}' = \mathbf{x} - \mathbf{t}$, as:

$$(\mathbf{x} - \mathbf{t})^T(\mathbf{t} \times \mathbf{x}) = 0$$

Coplanarity

The cross product $\mathbf{t} \times \mathbf{x}$ gives a vector orthogonal to the epipolar plane, but $\mathbf{x} - \mathbf{t}$ is lying on this plane, and the dot product between two orthogonal vectors gives $\mathbf{0}$.

Using the skew-symmetric matrix representation for the cross product, $\mathbf{t} \times \mathbf{x} = [\mathbf{t}]_\times \mathbf{x}$, the coplanarity equation becomes:

$$(\mathbf{x} - \mathbf{t})^T [\mathbf{t}]_\times \mathbf{x} = 0$$

Now, substitute the transformation $\mathbf{x} - \mathbf{t} = \mathbf{R}^T \mathbf{x}'$ derived from $\mathbf{x}' = \mathbf{R}(\mathbf{x} - \mathbf{t})$ by multiplying by $\mathbf{R}^T$ on the left:

$$(\mathbf{R}^T \mathbf{x}')^T [\mathbf{t}]_\times \mathbf{x} = 0 \quad \Longrightarrow \quad \mathbf{x}'^T \mathbf{R} [\mathbf{t}]_\times \mathbf{x} = 0$$

Which gives the *epipolar constraint equation*. Therefore, the essential matrix is defined as

$$\mathbf{E} = \mathbf{R}[\mathbf{t}]_\times$$

11.2.2 Properties of the essential matrix

Longuet-Higgins equation	$\boldsymbol{x}'^\top \mathbf{E} \boldsymbol{x} = 0$	
Epipolar lines	$\boldsymbol{x}^\top \boldsymbol{l} = 0$ $\boldsymbol{l}' = \mathbf{E}\boldsymbol{x}$	$\boldsymbol{x}'^\top \boldsymbol{l}' = 0$ $\boldsymbol{l} = \mathbf{E}^T \boldsymbol{x}'$
Epipoles	$\boldsymbol{e}'^\top \mathbf{E} = \mathbf{0}$	$\mathbf{E}\boldsymbol{e} = \mathbf{0}$

Properties of E

- It relates points in **normalized camera coordinates**.
- It has rank 2 ($\det(\mathbf{E}) = 0$ because $[\mathbf{t}]_\times$ is rank 2).
- It has 5 DOFs (3 for $\mathbf{R}$, 2 for translation direction).
- It has 2 equal non-zero singular values.

E vs. H

Both $\mathbf{E}$ and $\mathbf{H}$ are 3×3 matrices, but $\mathbf{E}$ maps a point to a line, while $\mathbf{H}$ maps a point to a point.

$$\mathbf{l}' = \mathbf{E}\mathbf{x} \qquad \mathbf{x}' = \mathbf{H}\mathbf{x}$$

11.3 The fundamental matrix

The *fundamental matrix* $\mathbf{F} \in \mathbb{R}^{3\times3}$ is a generalization of $\mathbf{E}$ for *uncalibrated* cameras (we don't know $\mathbf{K}$) $\Longrightarrow$ it operates directly on pixel coordinates.

11.3.1 Derivation

The relationship between pixel coordinates $\mathbf{p}$ and normalized camera coordinates $\mathbf{x}$ is $\mathbf{x} = \mathbf{K}^{-1}\mathbf{p}$. Substituting this into the essential matrix equation:

$$(\mathbf{K}'^{-1}\mathbf{p}')^T \mathbf{E} (\mathbf{K}^{-1}\mathbf{p}) = 0 \implies \mathbf{p}'^T (\mathbf{K}'^{-T} \mathbf{E} \mathbf{K}^{-1}) \mathbf{p} = 0$$

This gives the fundamental matrix equation:

$$\mathbf{p}'^T \mathbf{F} \mathbf{p} = 0$$

where the Fundamental Matrix is $\mathbf{F} = \mathbf{K}'^{-T}\mathbf{E}\mathbf{K}^{-1} = \mathbf{K}'^{-T}[\mathbf{t}]_\times \mathbf{R}\mathbf{K}^{-1}$.

11.3.2 Properties of the fundamental matrix

Just substitute the $\mathbf{E}$ with an $\mathbf{F}$ in the properties of $\mathbf{E}$, and $\mathbf{x}$ (points) with $\mathbf{p}$ (pixels).

Properties of F

- It relates points in **pixel coordinates**.
- It has rank 2 because $\det(\mathbf{F}) = 0$.
- It has 7 DOFs (8 DOFs as it is a projective transformation minus 1 because it is rank 2).
- It provides a "weak calibration" of the cameras.

F vs. E vs. H

Since $\mathbf{E}$ maps points to epipolar lines, $\mathbf{F}$ maps pixels to epipolar lines!

$$\mathbf{l}' = \mathbf{F}\mathbf{p}$$

We finally have that $\mathbf{H}$ is rank 3 with 8 DOFs, $\mathbf{F}$ is rank 2 with 7 DOFs, and $\mathbf{E}$ is rank 2 with 5 DOFs. Homography is a special case of $\mathbf{E}$ and $\mathbf{F}$ for planar scenes.

11.4 Estimating the fundamental matrix

We can estimate $\mathbf{F}$ from a set of corresponding points between two images, without knowing any camera parameters.

11.4.1 The 8-Point algorithm

Each point correspondence $(\mathbf{p}, \mathbf{p}')$ provides one linear equation for the 9 unknown elements of $\mathbf{F}$:

$$\begin{bmatrix} p'_x & p'_y & 1 \end{bmatrix} \begin{bmatrix} f_1 & f_2 & f_3 \\ f_4 & f_5 & f_6 \\ f_7 & f_8 & f_9 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix} = 0$$

which is

$$p_x p'_x f_{11} + p_y p'_x f_{12} + p'_x f_{13} + p_x p'_y f_{21} + p_y p'_y f_{22} + p'_y f_{23} + p_x f_{31} + p_y f_{32} + f_{33} = 0$$

Since $\mathbf{F}$ is defined up to scale, we need at least 8 point correspondences to solve for its elements. This leads to a homogeneous linear system $\mathbf{A}\mathbf{f} = \mathbf{0}$ solved by minimizing $||\mathbf{A}\mathbf{f}||^2$ subject to $||\mathbf{f}|| = 1$, whose solution is the smallest eigenvector of $\mathbf{A}^T\mathbf{A}$ corresponding to its smallest eigenvalue.

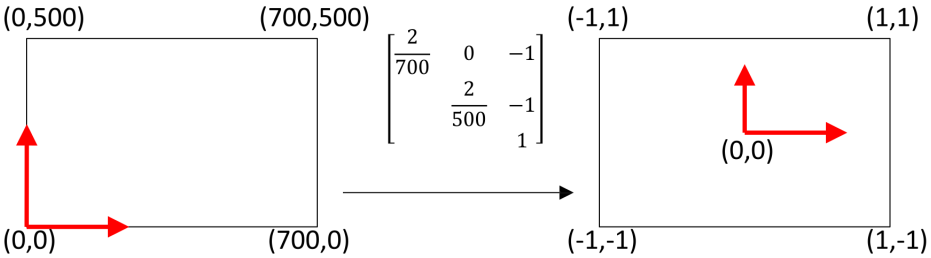
Numerical instability

The basic 8-point algorithm is very sensitive to noise because the columns of the matrix $\mathbf{A}$ can have vastly different magnitudes (e.g., columns for $p_x p'_x$ vs. columns for constant 1). This leads to poor numerical conditioning.

11.4.2 The normalized 8-Point algorithm

To overcome instability, we include a normalization step:

1. **Normalize points:** In each image, translate the points so their centroid is at the origin, then scale them so their average distance from the origin is $\sqrt{2}$. Let the transformation matrices be $\mathbf{T}$ and $\mathbf{T}'$.



2. **Compute F on normalized points:** Apply the basic 8-point algorithm to the normalized points $(\hat{\mathbf{p}}, \hat{\mathbf{p}}')$ to get an initial estimate $\hat{\mathbf{F}}$.

3. **Enforce rank constraint:** Since $\mathbf{F}$ must have rank 2, enforce this constraint on $\hat{\mathbf{F}}$. This is done by taking its SVD, $\hat{\mathbf{F}} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, setting the smallest singular value in $\mathbf{\Sigma}$ to zero to get $\mathbf{\Sigma}'$, and recomputing $\hat{\mathbf{F}}' = \mathbf{U}\mathbf{\Sigma}'\mathbf{V}^T$.

4. **Denormalize:** Transform the matrix back to the original coordinate system: $\mathbf{F} = \mathbf{T}'^T\hat{\mathbf{F}}'\mathbf{T}$.

11.5 Triangulation

Once camera parameters (projection matrices $\mathbf{P}, \mathbf{P}'$) and point correspondences $(\mathbf{x}, \mathbf{x}')$ are known, we apply *triangulation* to find the 3D location of the point $\mathbf{X}$.

11.5.1 Linear triangulation method

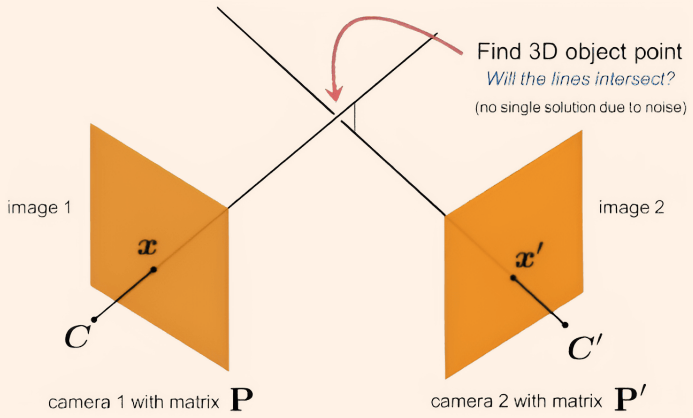
The projection equations are $\mathbf{x} \cong \mathbf{P}\mathbf{X}$ and $\mathbf{x}' \cong \mathbf{P}'\mathbf{X}$. Because these are homogeneous coordinate equations, they can be rewritten using the cross product to eliminate the unknown scale factor:

$$\mathbf{x} \times (\mathbf{P}\mathbf{X}) = \mathbf{0} \quad \text{and} \quad \mathbf{x}' \times (\mathbf{P}'\mathbf{X}) = \mathbf{0}$$

Each cross product provides two independent linear equations in the unknowns of $\mathbf{X} \in \mathbb{R}^4$, which is defined up to scale, so 3 independent parameters. Combining the equations from both views gives us a 4×3 homogeneous linear system $\mathbf{A}\mathbf{X} = \mathbf{0}$ to be solved for $\mathbf{X}$ using SVD.

The noise problem

In practice, due to noise in image measurements and camera parameters, the back-projected rays from $\mathbf{x}$ and $\mathbf{x}'$ will likely not intersect perfectly in 3D space. The linear SVD method finds a least-squares solution that minimizes an algebraic error, not the true geometric error. More advanced, non-linear optimization techniques like **bundle adjustment** are required to find the 3D point that minimizes the reprojection error back into the images.



Errata and Corrections to Chapters 1–11

The embedded PDF is included verbatim and cannot be edited in place. The following verified factual issues are therefore collected here.

E1 Essential matrix: convention inconsistency (§11.2–11.3)

The PDF derivation (pp. 58–59) adopts the pose convention $\mathbf{x}' = \mathbf{R}(\mathbf{x}-\mathbf{t})$, which correctly yields

$$\mathbf{E} = \mathbf{R}\,[\mathbf{t}]_\times.$$

However, the same section then writes the fundamental matrix as $\mathbf{F} = \mathbf{K}'^{-T}[\mathbf{t}]_\times \mathbf{R}\,\mathbf{K}^{-1}$, reversing the factor order $[\mathbf{t}]_\times \mathbf{R}$. This is internally inconsistent with the derivation above.

Correction

Consistent with the PDF's own derivation ($\mathbf{E} = \mathbf{R}[\mathbf{t}]_\times$), the correct fundamental matrix expression is:

$$\mathbf{F} = \mathbf{K}'^{-T}\,\mathbf{E}\,\mathbf{K}^{-1} = \mathbf{K}'^{-T}\,\mathbf{R}\,[\mathbf{t}]_\times\,\mathbf{K}^{-1}.$$

Exam caveat: the standard textbook convention (Hartley & Zisserman) defines the relative pose as $\mathbf{x}' = \mathbf{R}\mathbf{x} + \mathbf{t}$, giving $\mathbf{E} = [\mathbf{t}]_\times \mathbf{R}$. Both conventions are valid; always state explicitly which one you adopt.

E2 Histogram equalisation: normalisation mismatch (§1.1.2)

The CDF is defined as a normalised cumulative probability, $\mathrm{CDF}(r_k) = \sum_{i=0}^{k} p(r_i) \in [0, 1]$, yet the remapping formula uses the total pixel count N in the denominator, mixing normalised and unnormalised quantities.

Correction

Use *cumulative counts* $H(r_k) = \sum_{i=0}^{k} \#\{\text{pixels} = r_i\}$ throughout:

$$r'_k = \text{round}\left(\frac{H(r_k) - H_{\min}}{N - H_{\min}} (L-1) \right).$$

Equivalently, with the normalised CDF the denominator is $1 - \text{CDF}_{\min}$:

$$r'_k = \text{round}\left(\frac{\text{CDF}(r_k) - \text{CDF}_{\min}}{1 - \text{CDF}_{\min}} (L-1) \right).$$

E3 SIFT scale sampling: clarification (§6.1.1)

The notes associate $k = \sqrt{2}$ with the scale levels per octave without explanation. In Lowe's original formulation the scale multiplier is $k = 2^{1/s}$, where s is the number of *intervals* per octave. The default $s = 3$ gives $k = 2^{1/3} \approx 1.26$; the value $k = \sqrt{2}$ corresponds to $s = 2$. Both choices appear in course material; remember the general relation $k = 2^{1/s}$.

Chapter 12

Vision Transformers

12.1 From NLP to Vision: Motivation

Transformers were introduced in 2017 for natural language processing [6] and subsequently demonstrated to be highly effective in computer vision. The core component is the **self-attention** mechanism, which models all pairwise interactions within an input sequence in a single operation, with no locality bias.

12.2 Self-Attention

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- $Q \in \mathbb{R}^{n \times d_k}$: query matrix.
- $K \in \mathbb{R}^{m \times d_k}$: key matrix.
- $V \in \mathbb{R}^{m \times d_v}$: value matrix.
- The scaling $1/\sqrt{d_k}$ prevents the dot products from growing large and pushing softmax into regions of near-zero gradient.

12.2.1 Multi-Head Attention

Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q,\, KW_i^K,\, VW_i^V)$$

Each head attends to a different representation subspace; their outputs are concatenated and linearly projected by $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

12.2.2 Positional Encoding

Self-attention is permutation-equivariant: it has no intrinsic notion of token order. A **positional encoding** is added to the input embeddings to inject position information. The original formulation uses fixed sinusoidal functions; learnable positional embeddings are also common in vision models.

12.2.3 Transformer Encoder Block

Each encoder block applies, in sequence:

1. **Multi-head self-attention** with residual connection and layer normalisation.
2. **Feed-forward network** (two linear layers with a non-linearity) with residual connection and layer normalisation.

12.3 Vision Transformer (ViT)

ViT [7] applies a standard Transformer encoder directly to sequences of image patches.

12.3.1 ViT Pipeline

1. **Patch extraction**: divide the $H \times W$ image into non-overlapping patches of size $P \times P$; this yields $N = HW/P^2$ patches.
2. **Linear embedding**: flatten each $P \times P \times C$ patch into a vector and project to dimension d_{model}.
3. **Prepend class token**: a learnable `[CLS]` token is prepended; its final representation is used for classification.
4. **Add positional embeddings** (learnable 1-D encodings).
5. **Transformer encoder**: L blocks of multi-head self-attention + feed-forward network.
6. **Classification head**: MLP applied to the `[CLS] token` output.

ViT Key Properties

- Patches act as tokens, analogous to words in NLP.
- **Weak image-specific inductive bias**: unlike CNNs, ViT does not build in locality or translation equivariance. This is an asset at very large scale but a liability with limited data.
- Highly scalable and parallelisable (all-pairs attention is $O(N^2)$ in the number of patches).
- Requires large-scale pre-training (ImageNet-21k or JFT-300M) to match or outperform CNN baselines.

12.4 Swin Transformer

The Swin Transformer [8] introduces **shifted-window attention** to obtain linear complexity and a hierarchical, CNN-like feature pyramid.

12.4.1 Shifted-Window Attention

Self-attention is computed within local, non-overlapping windows of size $M \times M$ rather than globally. Between consecutive layers the windows are *shifted* by $(\lfloor M/2 \rfloor, \lfloor M/2 \rfloor)$ pixels, enabling cross-window connections without increasing cost.

Complexity Comparison

Method	Attention scope	Complexity
ViT	Global	$O(N^2)$
Swin	Window $(M \times M)$	$O(N)$

Swin Advantages

- Linear complexity $O(N)$ in the number of patches.
- Hierarchical, multi-scale representation suitable as a general-purpose backbone for detection and segmentation.
- Achieved state-of-the-art on COCO detection at the time of its introduction (2021).

12.5 DETR: End-to-End Object Detection with Transformers

DETR [9] reformulates object detection as a direct *set-prediction* problem, eliminating anchors and non-maximum suppression.

12.5.1 Architecture

1. A **CNN backbone** (e.g. ResNet-50) extracts a feature map of size $\frac{H}{32} \times \frac{W}{32} \times C$.
2. A **Transformer encoder** processes the flattened spatial features with positional encodings.
3. A **Transformer decoder** takes N_q learnable *object queries* and attends to encoder output, producing N_q embeddings.
4. Two **prediction heads** (MLP and linear) map each embedding to a bounding box and a class label (including "no object").
5. A **bipartite matching loss** (Hungarian algorithm) assigns each prediction uniquely to a ground-truth object during training.

DETR Advantages

- Fully end-to-end trainable: no hand-crafted anchor boxes, no NMS.
- Naturally extends to panoptic segmentation.
- Limitation: slow to converge (requires many training epochs); later variants (Deformable DETR, DINO) address this.

12.6 TimeSformer: Video Recognition

TimeSformer [16] extends the Transformer to video by factorising attention over the spatial and temporal dimensions separately (*divided space-time attention*), which is far cheaper than joint space-time self-attention.

1. **Temporal attention**: each patch attends to patches at the *same spatial location* across the other frames.
2. **Spatial attention**: each patch attends to all other patches *within the same frame*.

The two operations are applied sequentially within each Transformer block.

Chapter 13

Generative Adversarial Networks (GANs)

13.1 Formulation

A GAN [11] consists of two networks trained in an adversarial minimax game:

- **Generator** G: maps a latent vector $z \sim p_z$ to a synthetic sample $G(z)$.
- **Discriminator** D: outputs the probability that its input is a real sample rather than a generated one.

GAN Objective

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}\big[\log D(x)\big] + \mathbb{E}_{z\sim p_z}\big[\log(1 - D(G(z)))\big]$$

At the Nash equilibrium, G recovers p_{data} and $D \equiv 1/2$.

13.2 Training Dynamics and Common Failures

- **Mode collapse**: G learns to produce only a few modes of the data distribution, ignoring diversity.
- **Vanishing gradients**: when D is too strong, G receives near-zero gradients. The non-saturating loss $\max_G \mathbb{E}_z[\log D(G(z))]$ alleviates this.
- **Training instability**: the two-player game is inherently fragile; careful hyperparameter tuning and architectural choices are required.

13.3 Wasserstein GAN (WGAN)

WGAN [12] replaces the Jensen–Shannon divergence with the Wasserstein-1 (Earth Mover's) distance, which provides more stable gradients:

WGAN Objective

$$\min_G \max_{D \in \mathcal{D}_{\text{1-Lip}}} \mathbb{E}_{x \sim p_{\text{data}}}[D(x)] - \mathbb{E}_{z \sim p_z}[D(G(z))]$$

D is constrained to the class of 1-Lipschitz functions (enforced by weight clipping or gradient penalty in WGAN-GP).

13.4 Notable Architectures

- **DCGAN**: deep convolutional GAN; uses batch normalisation, strided convolutions, and ReLU/LeakyReLU — the canonical baseline.
- **Conditional GAN (cGAN)**: conditions both G and D on a class label or input image, enabling class-conditional generation.
- **CycleGAN**: unsupervised image-to-image translation using two generators and a *cycle-consistency loss* ($G(F(x)) \approx x$) to enforce bijective mapping without paired training data.
- **StyleGAN**: disentangles high-level attributes (style) from stochastic details (noise) via adaptive instance normalisation (AdaIN) applied at each resolution level; produces photorealistic faces.
- **BigGAN**: large-scale conditional GAN, achieving high FID/IS scores on ImageNet at 512×512.

13.5 Evaluation Metrics

- **Inception Score (IS)**: measures quality (high $p(y|x)$) and diversity (high $p(y)$ entropy) jointly. Higher is better; does not compare to real data distribution.
- **Fréchet Inception Distance (FID)**: measures the Fréchet distance between real and generated feature distributions (from Inception-v3):

  $$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{tr}\big(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\big)$$

  Lower is better; sensitive to mode collapse and mode dropping.

Chapter 14

Diffusion Models

14.1 Overview of Deep Generative Models

Model	Training	Sampling	Strengths
VAE	ELBO	single pass (fast)	stable, disentangled
GAN	adversarial	single pass (fast)	sharp images
Flow	exact likelihood	single pass	exact density
Diffusion	denoising score	iterative (slow)	best quality, diversity

14.2 DDPM: Denoising Diffusion Probabilistic Models

A diffusion model [10] defines a fixed *forward* process that gradually corrupts a real image with Gaussian noise until only pure noise remains, and learns a parametric *reverse* process that iteratively denoises, generating images from noise.

14.2.1 Forward Process

Forward (Noising) Process

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t \mathbf{I}\big), \qquad t = 1, \ldots, T$$

where $\{\beta_t\}$ is a fixed variance schedule ($0 < \beta_t < 1$, monotonically increasing).

Defining $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$, the reparameterisation trick gives a closed-form expression for any t:

Reparameterisation (Key Identity)

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

This allows sampling x_t at *any* timestep directly from x_0, without iterating through t steps.

14.2.2 Reverse Process

A neural network ϵ_θ (typically a U-Net) parameterises the reverse (denoising) transitions:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1};\ \mu_\theta(x_t, t),\ \Sigma_\theta(x_t, t)\big).$$

14.2.3 Training Objective

After deriving a variational lower bound (ELBO) and simplifying, the training objective reduces to a **noise-prediction** (denoising) loss:

DDPM Training Loss

$$L_{\text{simple}} = \mathbb{E}_{x_0,\,\epsilon,\,t}\Big[\ \big\|\epsilon - \epsilon_\theta(x_t, t)\big\|^2\ \Big]$$

The network learns to predict the noise ϵ that was added to produce x_t.

14.2.4 Training and Sampling Algorithms

Algorithm 1 DDPM Training

repeat
  Sample $x_0 \sim q(x_0)$
  Sample $t \sim \text{Uniform}(\{1, \dots, T\})$
  Sample $\epsilon \sim \mathcal{N}(0, \mathbf{I})$
  Compute $x_t \leftarrow \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$
  Take gradient step on $\nabla_\theta \left\|\epsilon - \epsilon_\theta(x_t, t)\right\|^2$
until converged

Algorithm 2 DDPM Sampling

Sample $x_T \sim \mathcal{N}(0, \mathbf{I})$
for $t = T, T-1, \dots, 1$ **do**
  $z \leftarrow \mathcal{N}(0, \mathbf{I})$ if $t > 1$, else $z \leftarrow 0$
  $x_{t-1} \leftarrow \dfrac{1}{\sqrt{\alpha_t}}\left(x_t - \dfrac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\, \epsilon_\theta(x_t, t)\right) + \sigma_t z$
end for
return x_0

Diffusion vs. GAN

- Diffusion models produce **higher diversity** and avoid mode collapse; training is **stable** (no adversarial game).
- Disadvantage: **slow sampling** ($T = 1000$ steps by default); mitigated by DDIM (deterministic sampling, ∼50 steps) and consistency models (1–2 steps).

14.3 Applications

Image generation (DALL·E 2, Stable Diffusion, Midjourney), video generation (Sora), text-to-image translation, image inpainting, super-resolution, and medical image synthesis.

Chapter 15

Graph Neural Networks (GNNs)

15.1 Graphs and Their Representations

A graph $G = (V, E)$ consists of a node set V ($|V| = n$) and an edge set E. Node features are collected in a matrix $\mathbf{H} \in \mathbb{R}^{n \times d}$.

- **Adjacency matrix A** $\in \{0,1\}^{n \times n}$: dense, $O(n^2)$ storage.
- **Adjacency list**: sparse representation, $O(n + |E|)$ storage.
- **Degree matrix D**: diagonal, $D_{ii} = \sum_j A_{ij}$.

Key structural properties:

- **Variable size**: the number of nodes and edges varies.
- **Non-Euclidean domain**: no intrinsic spatial coordinates.
- **Permutation invariance**: graph-level tasks must be invariant to node reordering.

15.2 Message Passing

Message passing is the core computational paradigm of GNNs. Node representations are iteratively updated by aggregating information from neighbours:

General Message Passing

$$h_u^{(k+1)} = \mathrm{UPDATE}^{(k)}\Big(h_u^{(k)},\ \mathrm{AGGREGATE}^{(k)}\big(\{\, h_v^{(k)} : v \in \mathcal{N}(u) \,\}\big)\Big)$$

AGGREGATE must be **permutation-invariant** (e.g. $\sum$, mean, max).

15.3 GNN Variants

15.3.1 Graph Convolutional Network (GCN)

GCN Layer

$$\mathbf{H}^{(l+1)} = \sigma\Big(\tilde{\mathbf{D}}^{-\frac{1}{2}}\,\tilde{\mathbf{A}}\,\tilde{\mathbf{D}}^{-\frac{1}{2}}\,\mathbf{H}^{(l)}\,\mathbf{W}^{(l)}\Big)$$

$\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ adds self-loops; $\tilde{\mathbf{D}}$ is the degree matrix of $\tilde{\mathbf{A}}$.

Symmetric normalisation $\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ prevents feature magnitude from growing with node degree.

15.3.2 Graph Attention Network (GAT)

GATs replace the fixed normalisation weights with *learned* attention coefficients:

GAT Attention

$$e_{ij} = \text{LeakyReLU}\big(\mathbf{a}^\top[\mathbf{W}h_i \,\|\, \mathbf{W}h_j]\big), \qquad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k\in\mathcal{N}_i}\exp(e_{ik})}$$

$$h_i' = \Big\|_{k=1}^{K} \sigma\left(\sum_{j\in\mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k h_j\right)$$

where $\|$ denotes concatenation over K attention heads.

15.3.3 Limitations

- **Over-smoothing**: with many layers, node features converge to indistinguishable values.
- **Structural sensitivity**: performance depends on graph quality.
- **Scalability**: full neighbourhood aggregation is expensive for large graphs; mini-batch sampling methods (GraphSAGE, ClusterGCN) are used in practice.

Chapter 16

Vision-Language Models (VLMs)

16.1 Motivation

Unimodal models are fundamentally limited: vision models extract spatial features but lack semantic reasoning, whereas language models handle syntax and context but lack perceptual grounding. Multimodal models jointly represent both modalities, enabling tasks such as visual question answering, image captioning, and zero-shot recognition.

16.2 Fusion Strategies

Strategy	Modalities fused at	Example
Early fusion	Input level	Single shared Transformer
Intermediate	Mid-network	Flamingo (gated cross-attention)
Late fusion	Output/prediction	Ensemble of unimodal models

16.3 CLIP: Contrastive Language-Image Pre-Training

CLIP [15] is trained on $\sim$400 million web-scraped image-text pairs using **contrastive learning**.

16.3.1 Architecture

- **Image encoder**: ViT or ResNet; maps images to ℓ_2-normalised embeddings $\mathbf{I}_k$.
- **Text encoder**: Transformer; maps tokenised captions to ℓ_2-normalised embeddings $\mathbf{T}_k$.
- Training maximises the cosine similarity of matching pairs and minimises it for non-matching pairs within a batch.

16.3.2 InfoNCE Loss

For a batch of N image-text pairs with temperature τ:

CLIP InfoNCE Loss

$$\mathcal{L}_{I\to T} = -\sum_{k=1}^{N} \log \frac{\exp(\mathbf{I}_k \cdot \mathbf{T}_k/\tau)}{\sum_{j=1}^{N} \exp(\mathbf{I}_k \cdot \mathbf{T}_j/\tau)}, \quad \mathcal{L}_{T\to I} = -\sum_{k=1}^{N} \log \frac{\exp(\mathbf{I}_k \cdot \mathbf{T}_k/\tau)}{\sum_{j=1}^{N} \exp(\mathbf{I}_j \cdot \mathbf{T}_k/\tau)}$$

$$\mathcal{L}_{\text{CLIP}} = \tfrac{1}{2}\big(\mathcal{L}_{I\to T} + \mathcal{L}_{T\to I}\big)$$

The loss forms an $N \times N$ similarity matrix; the diagonal is the set of positive pairs.

16.3.3 Zero-Shot Transfer

At inference, class names are embedded as text prompts (e.g. "a photo of a {class}"); the image is classified by finding the most similar text embedding. No task-specific fine-tuning is required.

Why CLIP Works

- **Zero annotation cost**: supervision comes from naturally occurring image-text pairs scraped from the web.
- **Scale**: hundreds of millions of pairs.
- **Strong zero-shot transfer**: flexible via prompt engineering.
- **Limitations**: no generative capability; limited counting and reasoning; sensitive to prompt design.

16.4 Modern VLMs

- **BLIP**: unifies understanding (contrastive + matching) and generation (language modelling) in a single pre-training framework.
- **Flamingo**: uses a frozen vision encoder and a frozen LLM connected by a Perceiver Resampler and gated cross-attention layers; strong few-shot learner.
- **Instruction tuning**: supervised fine-tuning on (system prompt, instruction + image, gold response) triples to improve instruction-following.
- **In-context learning (ICL)**: few-shot prompting without updating model weights.
- **Chain-of-thought (CoT)**: the model generates intermediate reasoning steps before the final answer, reducing hallucination.

Chapter 17

Medical Imaging

17.1 Imaging Modalities

- **X-ray**: electromagnetic radiation; high contrast for bone and lung pathology; dense tissue appears white, air appears black. Low cost, fast acquisition; 2-D projection.
- **CT (Computed Tomography)**: ionising radiation; 3-D reconstruction from many X-ray projections at different angles. Used for trauma, tumours, and vascular disease.
- **MRI (Magnetic Resonance Imaging)**: strong magnetic fields + radiofrequency pulses; best soft-tissue contrast (brain, abdomen); no ionising radiation; slow acquisition; sensitive to motion.
- **Ultrasound**: high-frequency sound waves; real-time imaging of organs, muscles, and foetuses; portable, low cost; operator- dependent, noisy (speckle).
- **PET (Positron Emission Tomography)**: functional imaging of metabolic activity (e.g. tumour staging); low spatial resolution; commonly fused with CT (PET-CT).

17.2 The Medical Data Gap

- **Volume**: medical datasets are orders of magnitude smaller than ImageNet.
- **Annotation**: requires expert knowledge (radiologists, pathologists); expensive and time-consuming.
- **Privacy**: strict regulation (GDPR, HIPAA) limits data sharing.
- **Domain characteristics**: low contrast, imaging artefacts, significant inter-scanner and inter-site variability.
- **Consequence**: models trained on one centre/scanner generalise poorly to others (*domain shift*).

Mitigation strategies: transfer learning from ImageNet or natural-image pre-trained models, data augmentation, federated learning, self-supervised pre-training, and meta-learning.

17.3 Segmentation Architectures

- **U-Net**: encoder–decoder with *skip connections* that pass feature maps from the encoder directly to the decoder, combining high-level semantics with fine-grained spatial detail. De facto standard for biomedical segmentation.
- **U-Net++**: nested skip connections providing multiple fusion paths at each scale.
- **TransUNet**: U-Net with a ViT encoder (applied at the bottleneck); captures long-range context at higher computational cost.
- **3D segmentation**: processes the full volumetric stack (CT/MRI volumes); captures inter-slice context and ensures 3-D consistency; greatly increases memory and training cost.

17.4 Loss Functions

Dice Loss (class-imbalance robust)

$$L_{\text{dice}} = 1 - \frac{1}{C}\sum_{c=0}^{C-1}\frac{2\sum_{n=1}^{N} t_n^c\, y_n^c}{\sum_{n=1}^{N}\left(t_n^c + y_n^c\right)}$$

Directly optimises the overlap between prediction and ground truth; insensitive to class frequency.

Focal Loss (hard-example mining)

$$L_{\text{focal}} = -\sum_{n=1}^{N}(1 - p_{t,n})^{\gamma}\,\log p_{t,n}, \qquad p_{t,n} = t_n \cdot y_n$$

Down-weights well-classified (easy) examples, focusing training on hard cases such as small lesions. Typically $\gamma = 2$.

In practice, the **combined Dice + Cross-Entropy loss** is commonly used: $L = L_{\text{dice}} + L_{\text{CE}}$.

17.5 Evaluation Metrics

Detection: IoU, Average Precision (AP), mean AP (mAP) at various IoU thresholds.

Segmentation:

- **Dice score** (DSC): $\frac{2|X \cap Y|}{|X|+|Y|}$ — equivalent to the F1 score; measures overlap.

- **Hausdorff Distance (HD)**:

$$\mathrm{HD}(X,Y) = \max\Big(\max_{x\in X}\min_{y\in Y}\|x-y\|,\ \max_{y\in Y}\min_{x\in X}\|x-y\|\Big)$$

  Measures the worst-case surface-to-surface deviation; sensitive to outliers (the 95th-percentile HD is often preferred).

17.6 Foundation Models for Medical Imaging

- **MedSAM / MedSAM2**: Segment Anything Model adapted to medical imaging; enables prompted 3-D segmentation from 2-D interactive inputs; reduces manual annotation burden by $> 85\%$.
- **UniMed-CLIP**: a unified VLM trained on 5.3M image-text pairs across six medical modalities (X-ray, CT, MRI, etc.); open-source.
- **UniChest**: federated multi-hospital training with shared global knowledge plus hospital-specific expert modules.

Chapter 18

Monocular Depth Estimation (MDE)

18.1 Problem Statement

Depth information is lost when a 3-D scene is projected onto a 2-D image. Recovering depth from a *single* image is therefore an inherently ill-posed problem: infinitely many 3-D scenes can produce the same 2-D image. The task is nevertheless tractable because images contain rich monocular cues.

18.1.1 Monocular Cues

- Linear perspective (converging parallel lines).
- Relative size (known objects appear smaller at greater depth).
- Texture gradient (texture elements become smaller with distance).
- Occlusion (near objects overlap far objects).
- Atmospheric haze (distant objects appear less saturated).

18.2 Learning Approaches

- **Fully supervised**: requires dense ground-truth depth maps (from LiDAR or structured light); best accuracy.
- **Semi-supervised**: uses sparse depth (e.g. LiDAR) combined with image reconstruction consistency.
- **Self-supervised / Unsupervised**: minimises photometric reprojection error between video frames or stereo pairs; no depth labels required; accuracy lower than supervised methods.

18.3 Evaluation Metrics

Error metrics (lower is better):

- Absolute Relative Error (AbsRel): $\frac{1}{|P|}\sum_i |p_i - g_i|/g_i$
- Root Mean Squared Error (RMSE): $\sqrt{\frac{1}{|P|}\sum_i (p_i - g_i)^2}$
- $\log_{10}$ error: $\frac{1}{|P|}\sum_i |\log_{10} p_i - \log_{10} g_i|$

Threshold Accuracy δ_i (higher is better)

$$\delta_i = \frac{1}{|P|}\left|\left\{p_i \;:\; \max\left(\tfrac{p_i}{g_i}, \tfrac{g_i}{p_i}\right) < 1.25^i\right\}\right|, \quad i \in \{1,2,3\}$$

The fraction of predicted depths p_i that agree with the ground truth g_i within a factor of 1.25^i.

18.4 Depth Anything v2

Depth Anything v2 follows a **synthetic-to-real pseudo-labelling** pipeline:

1. Train a *teacher* model on synthetic data (perfect ground truth, no domain-adaptation noise).
2. Generate pseudo-labels on a large set of real unlabelled images using the teacher.
3. Train a *student* model on the pseudo-labelled real data, achieving strong real-world generalisation.

18.5 Application: RGB-D for Deepfake Detection

Combining an RGB image with a depth map estimated by MDE exposes geometric inconsistencies introduced by face manipulation (blending boundaries, unnatural 3-D structure), improving deepfake detection accuracy compared to RGB-only approaches.

Chapter 19

Efficiency in Deep Learning

19.1 Efficient Training vs. Efficient Inference

- **Efficient training**: reduce wall-clock training time and optimise GPU/TPU utilisation (mixed-precision training, gradient checkpointing, distributed data parallelism).
- **Efficient inference**: minimise latency and resource consumption when deploying a trained model on new data (particularly relevant for edge/mobile deployment).

19.2 Techniques for Efficient Inference

- **Compact architectures**: design models with lower intrinsic complexity (e.g. linear-attention variants, MobileNet depthwise separable convolutions, EfficientNet compound scaling).
- **Pruning**: remove parameters (neurons, filters, or connections) that contribute least to task performance, based on magnitude or gradient-based importance scores.
- **Knowledge distillation**: train a small *student* to mimic the outputs (soft logits, intermediate features) of a large pre-trained *teacher*.
- **Quantisation**: reduce numerical precision of weights and activations (e.g. FP32 $\rightarrow$ INT8 or INT4), decreasing memory footprint and enabling integer arithmetic on hardware accelerators.
- **Structured vs. unstructured pruning**: structured pruning (removing whole filters/heads) yields hardware-friendly speedup; unstructured pruning (individual weights) achieves higher sparsity but requires sparse hardware support.

19.3 Efficiency Metrics

- **MACs**: multiply-accumulate operations — primary measure of model computational cost.

- **FLOPs**: floating-point operations ($\approx 2\times$ MACs).
- **Parameter count**: memory footprint of the model weights.
- **Latency**: wall-clock inference time on target hardware.
- **Throughput**: images processed per second.

19.4 Efficiency–Performance Trade-off

- **Pareto frontier**: the set of model configurations for which no other configuration is both more efficient *and* more accurate.
- **Efficiency Error Rate (EER)**: a composite metric that aggregates efficiency measures relative to a reference model R:

$$\text{EER} = \frac{1}{|I|} \sum_{i \in I} \frac{M_i}{R_i}$$

  where M_i is the value of metric i for the candidate model and R_i for the reference.

References

[1] Szeliski, R. (2022). *Computer Vision: Algorithms and Applications*, 2nd ed. Springer.

[2] Hartley, R., & Zisserman, A. (2004). *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press.

[3] Forsyth, D. A., & Ponce, J. (2011). *Computer Vision: A Modern Approach*, 2nd ed. Pearson.

[4] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[5] Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. *IJCV*, 60(2), 91–110.

[6] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*.

[7] Dosovitskiy, A., et al. (2021). An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale. *ICLR*.

[8] Liu, Z., et al. (2021). Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows. *ICCV*.

[9] Carion, N., et al. (2020). End-to-End Object Detection with Transformers. *ECCV*.

[10] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *NeurIPS*.

[11] Goodfellow, I., et al. (2014). Generative Adversarial Nets. *NeurIPS*.

[12] Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein GAN. *ICML*.

[13] Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *ICLR*.

[14] Veličković, P., et al. (2018). Graph Attention Networks. *ICLR*.

[15] Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. *ICML*.

[16] Bertasius, G., Wang, H., & Torresani, L. (2021). Is Space-Time Attention All You Need for Video Understanding? *ICML*.

[17] He, K., et al. (2016). Deep Residual Learning for Image Recognition. *CVPR*.

[18] Fei-Fei, L., & Perona, P. (2005). A Bayesian Hierarchical Model for Learning Natural Scene Categories. *CVPR*.